ARKA PCR — Rencana Upgrade ke Stack Modern (Next.js + MySQL)

> Panduan migrasi dari CodeIgniter 2 (PHP) ke **Next.js (Vuexy template)** + **Prisma ORM** + **MySQL**.
> Master data **Project**, **Unit/Equipment**, dan **Model** dikonsumsi dari API eksternal `ark-fleet`.
> Hanya data PCR-spesifik (komponen, policy, WO, SOS, inspeksi, kanibal) yang disimpan lokal.

Daftar Isi

1. Ringkasan Aplikasi Saat Ini
2. Masalah & Risiko Teknis (Legacy)
3. Stack Teknologi yang Digunakan
4. Arsitektur Baru
5. Integrasi Fleet API
6. Skema Database (Prisma Schema)
7. Pemetaan Fitur: Legacy → Modern
8. Struktur Folder Proyek
9. API Routes Mapping
10. Rencana Implementasi Bertahap
11. Detail Implementasi Per Modul
12. Keamanan & Auth
13. Estimasi Waktu
14. Catatan Migrasi Data

---

1. Ringkasan Aplikasi Saat Ini

ARKA PCR adalah sistem manajemen **Planned Component Replacement** untuk armada alat berat di industri pertambangan/konstruksi.

Domain Bisnis

```
ARKA PCR
├── Master Data (READ dari Fleet API — tidak disimpan lokal)
│   ├── Project     → GET 192.168.32.15/ark-fleet/api/projects
│   ├── Equipment   → GET 192.168.32.15/ark-fleet/api/equipments
│   └── Model       → embedded dalam response equipments (model, model_id, manufacture)
│
├── Master Data PCR (dikelola lokal di app ini)
│   ├── Component   (katalog part yang bisa diganti: Engine, Transmission, dll.)
│   └── ModelComponent / Commod  (mapping fleet_model_id ⇔ Component + kebijakan jam
ganti)
│
├── Operasional Harian (disimpan lokal, reference ke fleet_equipment_id)
│   ├── Hour Meter  (jam kerja unit, input rutin)
│   ├── PCR Forecast  (perencanaan ganti komponen per periode — NEW)
│   ├── Replacement/PCR  (Work Order penggantian komponen — actual)
│   ├── SOS  (Scheduled Oil Sampling — 47+ field lab analisis oli)
│   └── Inspection  (5 tipe: Filter Cut, Magnetic, Visual, TA2, Electronic Data)
│
├── Kondisi Komponen
```

```
  │   └── Condition  (Agregasi SOS + Inspection → skor A/B/C/X)
  │
  ├── PCR Forecast → Actual (alur bisnis)
  │   ├── 1. Forecasting (perencanaan per quarter/periode)
  │   ├── 2. Pengajuan BA PCR — Planner Foreman
  │   ├── 3. Pemeriksaan — Plant Supervisor
  │   ├── 4. Persetujuan — Project Manager + Plant Manager (paralel)
  │   ├── 5. Tinjauan Direksi — Operation Director, Finance Director, President Director
(paralel)
  │   └── 6. Realisasi PCR — record di `replacement` (modul PCR actual / aplikasi lama)
  │
  └── Cannibalization (BA / Berita Acara — terpisah dari BA PCR)
      ├── Kanibal  (detail baris REMOVE/INSTALL)
      └── Approval  (alur persetujuan L1 → L2 → L3)
```

Kalkulasi Kunci (Business Logic)

```
% Life Komponen = (HM Sekarang - HM saat ganti terakhir + Jam Comp terpasang) / Policy Jam
× 100

Forecast Ganti (hari) = (Policy - Jam Sekarang) / Rata-rata jam/hari (3 bulan terakhir)
```

Entitas — Lokal vs Fleet API

Sumber	Entitas	Keterangan
Fleet API	Project	`project_code`, `bowheer`, `location`
Fleet API	Equipment/Unit	`id`, `unit_no`, `description`, `project_code`, `model_id`, `model`, `manufacture`, `unitstatus`, dll.
Fleet API	Model	Embedded di Equipment: `model_id`, `model`, `manufacture`, `plant_group`
Lokal	Component	Katalog part, dikelola di ARKA PCR
Lokal	ModelComponent	Mapping `fleet_model_id` ↔ Component + policy jam
Lokal	HourMeter	Referensi ke `fleet_equipment_id`
Lokal	Replacement	PCR Work Order (actual), referensi ke `fleet_equipment_id`
Lokal	PcrForecast	Perencanaan PCR per periode — bisa di-convert ke `replacement`
Lokal	PcrForecastApproval	Approval BA PCR per role (PS→PM/PLM→Direksi) — terpisah dari BA kanibal L1/L2/L3
Lokal	SosRecord	Oil sampling, referensi ke `fleet_equipment_id`
Lokal	Inspection	Inspeksi, referensi ke `fleet_equipment_id`
Lokal	ComponentCondition	Kondisi final per komponen
Lokal	BaDocument	Dokumen kanibal, referensi `project_code`
Lokal	Kanibal	Detail baris BA, referensi ke `fleet_equipment_id`

Sumber	Entitas	Keterangan
Lokal	User	Akun login ARKA PCR
Lokal	FleetEquipmentCache	Snapshot equipment dari Fleet API (sync periodik)
Lokal	LegacyUnitMapping	Mapping `id_unit` lama → `fleet_equipment_id` (migrasi)
Lokal	BaApproval	Approval BA normalisasi (L1/L2/L3) — kolom flat di `ba` tetap ada untuk kompatibilitas

2. Masalah & Risiko Teknis (Legacy)

#	Masalah	Tingkat Risiko	Keterangan
1	**CodeIgniter 2** — EOL sejak 2015	🔴 Kritis	Tidak ada security patch
2	**Password plaintext** di database	🔴 Kritis	Tidak di-hash sama sekali
3	**SQL Injection** — query interpolasi langsung	🔴 Kritis	`$bagianWhere`, `$st` langsung masuk query
4	**CSRF & XSS disabled** di config	🔴 Kritis	Rentan attack
5	**Login bug** — `$level` & `$kode_project` undefined	🟠 Tinggi	Fungsi login bisa error
6	`truncate()` **tanpa konfirmasi**	🟠 Tinggi	Bisa hapus semua data unit
7	`welcome.php` **pakai raw mysqli** di luar CI	🟡 Sedang	Inconsistent DB layer
8	**PHPExcel** — deprecated, tidak compatible PHP 8+	🟡 Sedang	Harus diganti ExcelJS
9	**Schema drift** — `mod` vs `commod`, missing tables di SQL dump	🟡 Sedang	Dokumentasi tidak akurat
10	**No migrations** — schema dikelola manual	🟡 Sedang	Sulit track perubahan
11	**No caching** untuk data master yang jarang berubah	🟢 Rendah	Performa

3. Stack Teknologi yang Digunakan

Core Stack

Layer	Teknologi	Keterangan
Framework	Next.js (App Router)	Versi Vuexy template yang ada — **tidak perlu install baru**
UI Template	Vuexy (React/Next.js)	Template berlisensi yang sudah dimiliki
Language	TypeScript	Type safety
Database	MySQL 8.x	Kompatibel dengan data yang ada
ORM	Prisma 6.x	Type-safe queries, migration otomatis
Auth	NextAuth.js (Auth.js v5)	Credentials provider + JWT
HTTP Client	Axios / native fetch	Konsumsi Fleet API
Server Cache	Next.js `unstable_cache` / `cache()`	Cache response Fleet API

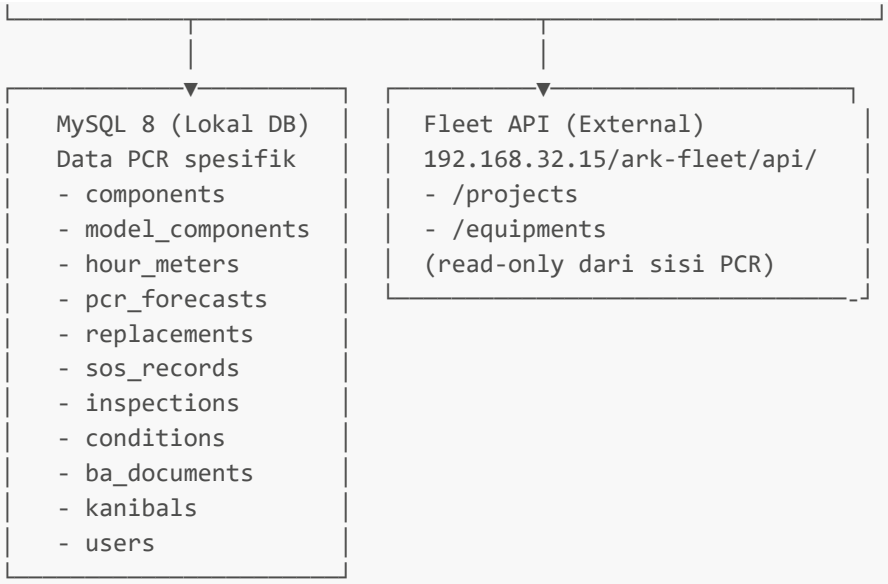
Layer	Teknologi	Keterangan
Forms	React Hook Form + Zod	Validasi type-safe
Excel	ExcelJS	Pengganti PHPExcel
Tables	TanStack Table	Atau gunakan komponen Vuexy
Charts	Recharts / ApexCharts	Atau gunakan yang sudah ada di Vuexy

Development Tools

Tool	Tujuan
`pnpm` / `npm`	Sesuaikan dengan package manager Vuexy
`eslint` + `prettier`	Code quality
`vitest`	Unit test kalkulasi bisnis

4. Arsitektur Baru





Prinsip Utama

- **Fleet API = source of truth** untuk Project, Equipment, Model — tidak pernah dimodifikasi dari sini
- **Data PCR lokal** menyimpan referensi `fleet_equipment_id` (INT) dan `project_code` (STRING) sebagai foreign reference ke fleet API
- Data fleet di-**cache di server** (Next.js cache) agar tidak membebani Fleet API setiap request
- **Vuexy template** digunakan sebagai fondasi UI — komponen tabel, form, dan layout sudah tersedia

---

5. Integrasi Fleet API

Endpoint yang Digunakan

Endpoint	Method	Kegunaan di ARKA PCR
`/api/projects`	GET	Dropdown project, filter data, scope user
`/api/equipments`	GET	Daftar unit, autocomplete, mapping ke PCR data
`/api/equipments/:id`	GET	Detail unit untuk halaman PCR unit

Struktur Response Fleet API

`/api/projects`

```
{
  "project_code": "000H",
  "bowheer": "Head Office",
  "location": "Balikpapan"
}
```

`/api/equipments`

```
{
  "id": 229,
```

```
  "unit_no": "AC 001",
  "description": "Air Compressor Yanmar TF55",
  "active_date": "2004-08-17",
  "serial_no": "Comp type LWU 050",
  "project_code": "008C",
  "project_id": 3,
  "plant_group": "Compressor",
  "plant_group_id": 10,
  "model": "TF55",
  "model_id": 271,
  "manufacture": "Yanmar",
  "unitstatus": "IN-ACTIVE",
  "unitstatus_id": 2,
  "asset_category": "Mayor",
  "asset_category_id": 1,
  "plant_type": "SUPPORT",
  "plant_type_id": 3
}
```

TypeScript Types untuk Fleet API

Letakkan di `types/fleet-api.ts`:

```typescript
// types/fleet-api.ts

export interface FleetProject {
  project_code: string;
  bowheer: string;
  location: string;
}

export interface FleetEquipment {
  id: number;
  unit_no: string;
  description: string;
  active_date: string | null;
  nomor_polisi: string | null;
  serial_no: string | null;
  chasis_no: string | null;
  engine_model: string | null;
  machine_no: string | null;
  bahan_bakar: string | null;
  warna: string | null;
  capacity: string | null;
  remarks: string | null;
  project_code: string;
  project_id: number;
  plant_group: string;
  plant_group_id: number;
  model: string;
  model_id: number;
  manufacture: string;
  unitstatus: "ACTIVE" | "IN-ACTIVE" | string;
  unitstatus_id: number;
  asset_category: string;
  asset_category_id: number;
```

```
  plant_type: string;
  plant_type_id: number;
}

// Tipe ringkas untuk dropdown/lookup
export interface FleetEquipmentOption {
  id: number;
  unit_no: string;
  description: string;
  project_code: string;
  model_id: number;
  model: string;
  manufacture: string;
  unitstatus: string;
}
```

Fleet API Client

Letakkan di `lib/fleet-api/client.ts`:

```
// lib/fleet-api/client.ts
import { unstable_cache } from "next/cache";
import type { FleetProject, FleetEquipment } from "@/types/fleet-api";

const FLEET_API_BASE =
  process.env.FLEET_API_URL ?? "http://192.168.32.15/ark-fleet/api";

async function fetchFleet<T>(path: string): Promise<T> {
  const res = await fetch(`${FLEET_API_BASE}${path}`, {
    headers: { Accept: "application/json" },
    // Fallback jika unstable_cache belum dipakai
    next: { revalidate: 600 }, // 10 menit
  });
  if (!res.ok) throw new Error(`Fleet API error: ${res.status} ${path}`);
  return res.json();
}

// Cache 10 menit di server
export const getProjects = unstable_cache(
  async (): Promise<FleetProject[]> => {
    return fetchFleet<FleetProject[]>("/projects");
  },
  ["fleet-projects"],
  { revalidate: 600, tags: ["fleet-projects"] },
);

export const getEquipments = unstable_cache(
  async (): Promise<FleetEquipment[]> => {
    return fetchFleet<FleetEquipment[]>("/equipments");
  },
  ["fleet-equipments"],
  { revalidate: 600, tags: ["fleet-equipments"] },
);

export async function getEquipmentsByProject(
  projectCode: string,
```

```
): Promise<FleetEquipment[]> {
  const all = await getEquipments();
  if (projectCode === "000H") return all;
  return all.filter((e) => e.project_code === projectCode);
}

export async function getEquipmentById(
  id: number,
): Promise<FleetEquipment | null> {
  const all = await getEquipments();
  return all.find((e) => e.id === id) ?? null;
}

// Ambil daftar model unik dari equipments (untuk mapping commod)
export async function getUniqueModels(): Promise<
  Array<{
    model_id: number;
    model: string;
    manufacture: string;
    plant_group: string;
  }>
> {
  const equipments = await getEquipments();
  const map = new Map<number, (typeof equipments)[0]>();
  for (const eq of equipments) {
    if (!map.has(eq.model_id)) map.set(eq.model_id, eq);
  }
  return Array.from(map.values()).map((e) => ({
    model_id: e.model_id,
    model: e.model,
    manufacture: e.manufacture,
    plant_group: e.plant_group,
  }));
}
```

Fleet API Proxy Route (untuk Client Components)

Beberapa komponen Vuexy berjalan di browser dan perlu data fleet. Buat proxy di:

```
// app/api/fleet/projects/route.ts
import { getProjects } from "@/lib/fleet-api/client";
import { auth } from "@/lib/auth";
import { NextResponse } from "next/server";

export async function GET() {
  const session = await auth();
  if (!session)
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  const projects = await getProjects();
  return NextResponse.json(projects);
}
```

```
// app/api/fleet/equipments/route.ts
import { getEquipmentsByProject } from "@/lib/fleet-api/client";
```

```
import { auth } from "@/lib/auth";
import { NextResponse } from "next/server";

export async function GET() {
  const session = await auth();
  if (!session)
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  const projectCode = session.user.projectCode ?? "000H";
  const equipments = await getEquipmentsByProject(projectCode);
  return NextResponse.json(equipments);
}
```

Project Scoping (kode `000H` = lihat semua)

```
// lib/utils/project-scope.ts
import type { Session } from "next-auth";
import type { FleetEquipment } from "@/types/fleet-api";

export const HEAD_OFFICE_CODE = "000H";

export function isHeadOffice(session: Session): boolean {
  return session.user.projectCode === HEAD_OFFICE_CODE;
}

// Filter equipment berdasarkan project user
export function filterByProject(
  equipments: FleetEquipment[],
  session: Session,
): FleetEquipment[] {
  if (isHeadOffice(session)) return equipments;
  return equipments.filter((e) => e.project_code === session.user.projectCode);
}

// Untuk query Prisma — filter berdasarkan project_code
export function getPrismaProjectFilter(session: Session): {
  projectCode?: string;
} {
  if (isHeadOffice(session)) return {};
  return { projectCode: session.user.projectCode };
}
```

---

6. Skema Database (Prisma Schema) — v2

> **Penting:** Tabel `project`, `unit`, dan `model` **tidak ada** karena diambil dari Fleet API.
> Data lokal menyimpan referensi `fleetEquipmentId` (INT = `id` dari Fleet API) dan `projectCode` (VARCHAR = `project_code` dari Fleet API).
> **Semua field legacy dipertahankan** — perbaikan v2 menambah constraint, cache Fleet, audit trail, dan normalisasi approval **tanpa menghapus kolom lama**.

Perbaikan v2 (ringkas)

Area	Masalah v1	Perbaikan v2

Area	Masalah v1	Perbaikan v2
Fleet API	Referensi tanpa jangkar lokal	Tabel `fleet_equipment_cache` + sync job
Migrasi	Mapping `id_unit` tidak terdokumentasi	Tabel `legacy_unit_mapping`
`commod`	Relasi `kanibals` invalid, bisa duplikat	Hapus relasi salah, `@@unique([id_model, id_comp])`
`kanibal.id_rep`	`0` = no WO, FK invalid	`NULL` = no WO (migrate `0` → `NULL`)
`ba` / `user`	Kolom `id_project` (INT) dipakai string	Rename ke `project_code`
HM / life	`Float` — error pembulatan	`Decimal` untuk kalkulasi
`wo_no`, `mr_no`, dll	`Int` — format dokumen terbatas	`VARCHAR(30)`
`condition`	Hanya 1 kolom overall	+ rating per sumber (SOS, FC, MPS, VI, TA2, ED)
Approval	Flat column tanpa timestamp/FK	Tabel `ba_approval` + kolom legacy tetap
Denormalisasi	`unit_no`/`project_code` bisa basi	+ `snapshot_at` (snapshot saat transaksi)
Audit	Tidak ada	`created_at`, `updated_at`, `deleted_at`, `created_by`
Index	Kurang untuk queue/report	Index composite ditambah

Perbandingan Nama Tabel: Legacy → Prisma Model

Tabel Lama	Prisma Model	Keterangan
(baru)	`FleetEquipmentCache`	Snapshot equipment dari Fleet API
(baru)	`LegacyUnitMapping`	Mapping migrasi `id_unit` → fleet id
`comp`	`Comp`	Tetap sama
`mod` / `commod`	`Commod`	`mod` di SQL dump 2016, `commod` di live DB
`hm`	`Hm`	Tetap sama + FK ke cache
`replacement`	`Replacement`	Tetap sama + relasi ke forecast
(baru)	`PcrForecast`	Perencanaan PCR per periode/quarter
(baru)	`PcrForecastApproval`	Approval BA PCR per role organisasi (6 level)
`sos`	`Sos`	47 kolom + `lab_results_json` opsional
`inspection`	`Inspection`	Tetap sama
`condition`	`Condition`	+ rating per sumber inspeksi/SOS
`ba`	`Ba`	Kolom approval flat + relasi `BaApproval`
(baru)	`BaApproval`	Approval normalisasi L1/L2/L3
`kanibal`	`Kanibal`	`id_rep` nullable
`ba_caused`	`BaCaused`	Tetap sama
`ba_action`	`BaAction`	Tetap sama
`ba_status`	`BaComponentStatus`	Lookup kondisi komponen saat BA

Tabel Lama	Prisma Model	Keterangan
user	User	+ sign, is_active, full_name

Salin ke prisma/schema.prisma:

```
// prisma/schema.prisma — v2

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

// ═══════════════════════════════════════════════════
// FLEET API — CACHE & MIGRASI
// ═══════════════════════════════════════════════════

/// Snapshot equipment dari Fleet API — di-sync periodik (cron/job)
model FleetEquipmentCache {
  fleetEquipmentId Int      @id @map("fleet_equipment_id")
  unitNo           String   @db.VarChar(20) @map("unit_no")
  description      String?  @db.VarChar(200)
  projectCode      String   @db.VarChar(10) @map("project_code")
  fleetModelId     Int      @map("fleet_model_id")
  modelName        String?  @db.VarChar(50)  @map("model_name")
  manufacture      String?  @db.VarChar(50)
  unitStatus       String?  @db.VarChar(20)  @map("unit_status")
  syncedAt         DateTime @default(now())  @map("synced_at")

  hmRecords    Hm[]
  replacements Replacement[]
  pcrForecasts PcrForecast[]
  sosRecords   Sos[]
  inspections  Inspection[]
  conditions   Condition[]
  kanibals     Kanibal[]

  @@index([unitNo])
  @@index([projectCode])
  @@index([fleetModelId])
  @@map("fleet_equipment_cache")
}

/// Mapping sekali pakai: id_unit legacy → fleet equipment id
model LegacyUnitMapping {
  legacyUnitId     Int      @id @map("legacy_unit_id")
  fleetEquipmentId Int      @map("fleet_equipment_id")
  legacyUnitNo     String?  @db.VarChar(20) @map("legacy_unit_no")
  mappedAt         DateTime @default(now())  @map("mapped_at")

  @@index([fleetEquipmentId])
  @@map("legacy_unit_mapping")
}
```

```
// ═══════════════════════════════════════════════════════
// MASTER DATA PCR
// ═══════════════════════════════════════════════════════

model Comp {
  idComp   Int     @id @default(autoincrement()) @map("id_comp")
  compDesc String  @db.VarChar(50)               @map("comp_desc")
  compType String? @db.VarChar(50)               @map("comp_type")
  status   String  @default("Active") @db.VarChar(10) // "Active" | "Inactive"

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  deletedAt DateTime? @map("deleted_at")

  commods Commod[]

  @@map("comp")
}

model Commod {
  idMod        Int      @id @default(autoincrement()) @map("id_mod")
  fleetModelId Int      @map("id_model") // = model_id dari Fleet API
  idComp       Int      @map("id_comp")
  policy       Int?
  price        Decimal? @db.Decimal(15, 2)
  lifeType     String?  @db.VarChar(20) @map("life_type") // "Hour" | "Calendar"

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")

  comp         Comp          @relation(fields: [idComp], references: [idComp])
  replacements Replacement[]
  pcrForecasts PcrForecast[]
  sos          Sos[]
  inspections  Inspection[]
  conditions   Condition[]

  @@unique([fleetModelId, idComp], name: "uq_commod_model_comp")
  @@index([fleetModelId])
  @@map("commod")
}

// ═══════════════════════════════════════════════════════
// OPERASIONAL
// ═══════════════════════════════════════════════════════

model Hm {
  idHm             Int      @id @default(autoincrement()) @map("id_hm")
  fleetEquipmentId Int      @map("id_unit")
  hmUnit           Decimal  @db.Decimal(12, 2) @map("hm_unit")
  whDay            Int      @map("wh_day")
  dateHm           DateTime @db.Date @map("date_hm")
  isOfficial       Boolean  @default(true) @map("is_official") // false jika duplikat
legacy

  // Snapshot saat input (denormalized — jangan di-update retroaktif)
  unitNo      String   @db.VarChar(20) @map("unit_no")
  projectCode String   @db.VarChar(10) @map("project_code")
```

```
  snapshotAt  DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  equipment FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])
  creator   User?               @relation("HmCreatedBy", fields: [createdBy], references:
[idUser])

  @@index([fleetEquipmentId, dateHm, idHm])
  @@index([projectCode])
  @@map("hm")
}

model Replacement {
  idRep            Int       @id @default(autoincrement()) @map("id_rep")
  repDate          DateTime  @db.Date @map("rep_date")
  lastRepDate      DateTime? @db.Date @map("last_rep_date")
  fleetEquipmentId Int       @map("id_unit")
  idMod            Int       @map("id_mod")
  hmRep            Decimal   @db.Decimal(12, 2) @map("hm_rep")
  lastHmRep        Decimal   @default(0) @db.Decimal(12, 2) @map("last_hm_rep")
  woNo             String?   @db.VarChar(30) @map("wo_no")
  woDate           DateTime? @db.Date @map("wo_date")
  woStatus         String    @default("OPEN") @db.VarChar(10) @map("wo_status") // "OPEN"
| "CLOSE"
  woEndDate        DateTime? @db.Date @map("wo_end_date")
  compHour         Int?      @map("comp_hour")
  compLife         Decimal   @default(0) @db.Decimal(12, 2) @map("comp_life")    //
snapshot saat close
  lifePercent      Decimal   @default(0) @db.Decimal(6, 2)  @map("life_percent") //
snapshot saat close
  lifeCalculatedAt DateTime? @map("life_calculated_at") // kapan comp_life terakhir
dihitung
  compCond         String    @default("A") @db.VarChar(1) @map("comp_cond")
  remarks          String    @default("") @db.Text
  report           String?   @db.VarChar(255)

  unitNo      String   @db.VarChar(20) @map("unit_no")
  projectCode String   @db.VarChar(10) @map("project_code")
  snapshotAt  DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  commod    Commod                @relation(fields: [idMod], references: [idMod])
  equipment FleetEquipmentCache   @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])
  creator   User?                 @relation("ReplacementCreatedBy", fields: [createdBy],
references: [idUser])
  kanibals  Kanibal[]
  forecast  PcrForecast?

  @@index([fleetEquipmentId, idMod, woStatus])
```

```
  @@index([woNo])
  @@index([projectCode])
  @@map("replacement")
}

/// Perencanaan PCR per periode — bisa di-convert ke `replacement` (actual WO)
model PcrForecast {
  idForecast       Int       @id @default(autoincrement()) @map("id_forecast")
  fleetEquipmentId Int       @map("id_unit")
  idMod            Int       @map("id_mod")

  // Snapshot saat buat/refresh (sesuai kolom spreadsheet)
  modelName      String?  @db.VarChar(50)  @map("model_name")       // Model Unit
  unitNo         String   @db.VarChar(20)  @map("unit_no")          // No Unit
  projectCode    String   @db.VarChar(10)  @map("project_code")     // Site
  compDesc       String?  @db.VarChar(50)  @map("comp_desc")        // Component
  hmComponent    Decimal  @db.Decimal(12, 2) @map("hm_component")   // HM Component
  policy         Int?                                              // Policy
  lifePercent    Decimal  @db.Decimal(6, 2)  @map("life_percent")   // Life Time Component
%
  ratingSos      String?  @db.VarChar(1)   @map("rating_sos")       // Rating S.O.S
(A/B/C/X)
  priceComponent Decimal? @db.Decimal(15, 2) @map("price_component") // Price Component
  snapshotAt     DateTime @default(now())  @map("snapshot_at")

  // Perencanaan
  planPeriod DateTime @db.Date @map("plan_period") // Plan Periode (May-26 → 2026-05-01)
  quarter    String   @db.VarChar(2)                // Q1 | Q2 | Q3 | Q4

  // BA PCR — terpisah dari tabel `ba` (kanibal)
  noBaPcr       String?   @db.VarChar(30) @map("no_ba_pcr")
  baPcrStatus   String    @default("PENDING") @db.VarChar(15) @map("ba_pcr_status")
  // PENDING | SUBMITTED | IN_REVIEW | APPROVED | REJECTED
  statusBaPcr   String?   @db.VarChar(100) @map("status_ba_pcr")  // "Wait Plant Spv",
"Wait Project Manager"
  baSubmittedAt DateTime? @db.Date @map("ba_submitted_at")         // Tanggal Pengajuan BA
PCR
  submittedBy   Int?      @map("submitted_by")                     // Planner Foreman
(pengaju)

  // Status actual (spreadsheet STATUS PCR)
  status     String    @default("OPEN") @db.VarChar(10) // OPEN | CLOSED | CANCELLED
  actionDate DateTime? @db.Date @map("action_date")      // ACTION DATE PCR
  poNumber   String?   @db.VarChar(30) @map("po_number") // PO
  remark     String?   @db.Text                          // REMARK

  // Link ke WO actual setelah convert
  idRep  Int?   @unique @map("id_rep")
  source String @default("MANUAL") @db.VarChar(20) // MANUAL | EXCEL | AUTO

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  commod      Commod              @relation(fields: [idMod], references: [idMod])
  equipment   FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])
  replacement Replacement?        @relation(fields: [idRep], references: [idRep])
```

```
  creator     User?               @relation("PcrForecastCreatedBy", fields: [createdBy],
references: [idUser])
  submitter   User?               @relation("PcrForecastSubmittedBy", fields:
[submittedBy], references: [idUser])
  approvals   PcrForecastApproval[]

  @@index([projectCode, quarter, planPeriod])
  @@index([fleetEquipmentId, idMod, status])
  @@index([status, baPcrStatus])
  @@map("pcr_forecast")
}

/// Approval BA PCR — workflow organisasi (bukan L1/L2/L3 kanibal)
/// Tahap 1: PS | Tahap 2 (paralel): PM + PLM | Tahap 3 (paralel): OD + FD + PD
model PcrForecastApproval {
  idForecastApproval Int       @id @default(autoincrement()) @map("id_forecast_approval")
  idForecast         Int       @map("id_forecast")
  level              String    @db.VarChar(5)
  // PS = Plant Supervisor (pemeriksaan)
  // PM = Project Manager | PLM = Plant Manager
  // OD = Operation Director | FD = Finance Director | PD = President Director
  stepOrder          Int       @map("step_order") // 1 | 2 | 3
  status             String    @default("PENDING") @db.VarChar(10) // PENDING | APPROVED |
REJECTED
  approverLabel      String?   @db.VarChar(100) @map("approver_label")
  approvedBy         Int?      @map("approved_by")
  approvedAt         DateTime? @map("approved_at")
  note               String?   @db.Text

  forecast PcrForecast @relation(fields: [idForecast], references: [idForecast], onDelete:
Cascade)
  approver User?       @relation("PcrForecastApprover", fields: [approvedBy], references:
[idUser])

  @@unique([idForecast, level], name: "uq_forecast_approval_level")
  @@index([level, status])
  @@map("pcr_forecast_approval")
}

model Sos {
  idSos            Int      @id @default(autoincrement()) @map("id_sos")
  fleetEquipmentId Int      @map("id_unit")
  idMod            Int      @map("id_mod")
  type             String   @default("SOS") @db.VarChar(10)
  sampleDate       DateTime @db.Date @map("sample_date")
  labName          String?  @db.VarChar(100) @map("lab_name")
  labNo            String?  @db.VarChar(50)  @map("lab_no")
  oilType          String?  @db.VarChar(100) @map("oil_type")
  hOil             Int?     @map("h_oil")
  hUnit            Int?     @map("h_unit")
  evalCode         String?  @db.VarChar(5)  @map("eval_code")
  recommendation   String?  @db.Text
  oilChange        Boolean? @default(false) @map("oil_change")
  oilAdded         Boolean? @default(false) @map("oil_added")

  fe    Decimal? @db.Decimal(8, 3)
  cu    Decimal? @db.Decimal(8, 3)
  cr    Decimal? @db.Decimal(8, 3)
  si    Decimal? @db.Decimal(8, 3)
```

```
  al    Decimal? @db.Decimal(8, 3)
  ni    Decimal? @db.Decimal(8, 3)
  sn    Decimal? @db.Decimal(8, 3)
  pb    Decimal? @db.Decimal(8, 3)
  pq    Decimal? @db.Decimal(8, 3)
  soot  Decimal? @db.Decimal(8, 3)
  oxid  Decimal? @db.Decimal(8, 3)
  nitr  Decimal? @db.Decimal(8, 3)
  sox   Decimal? @db.Decimal(8, 3)
  p4um  Decimal? @db.Decimal(8, 3) @map("4um")
  p6um  Decimal? @db.Decimal(8, 3) @map("6um")
  p14um Decimal? @db.Decimal(8, 3) @map("14um")
  p15um Decimal? @db.Decimal(8, 3) @map("15um")
  iso4406 String? @db.VarChar(20)
  iso14   String? @db.VarChar(20)
  iso6    String? @db.VarChar(20)
  ca   Decimal? @db.Decimal(8, 3)
  zn   Decimal? @db.Decimal(8, 3)
  mo   Decimal? @db.Decimal(8, 3)
  bo   Decimal? @db.Decimal(8, 3)
  p    Decimal? @db.Decimal(8, 3)
  na   Decimal? @db.Decimal(8, 3)
  k    Decimal? @db.Decimal(8, 3)
  mg   Decimal? @db.Decimal(8, 3)
  visc     Decimal? @db.Decimal(8, 3)
  tbn      Decimal? @db.Decimal(8, 3)
  tan      Decimal? @db.Decimal(8, 3)
  gly      Decimal? @db.Decimal(8, 3)
  water    Decimal? @db.Decimal(8, 3)
  dilution Decimal? @db.Decimal(8, 3)

  labResultsJson Json? @map("lab_results_json") // field lab tambahan / future

  unitNo      String   @db.VarChar(20) @map("unit_no")
  projectCode String   @db.VarChar(10) @map("project_code")
  snapshotAt  DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  commod    Commod              @relation(fields: [idMod], references: [idMod])
  equipment FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])
  creator   User?               @relation("SosCreatedBy", fields: [createdBy], references:
[idUser])

  @@index([fleetEquipmentId, idMod, sampleDate])
  @@index([projectCode])
  @@map("sos")
}

model Inspection {
  idIns            Int      @id @default(autoincrement()) @map("id_ins")
  fleetEquipmentId Int      @map("id_unit")
  idMod            Int      @map("id_mod")
  type             String   @db.VarChar(10) // "FC" | "MPS" | "VI" | "TA2" | "ED"
  insDate          DateTime @db.Date @map("ins_date")
```

```
  insHm            Int?     @map("ins_hm")
  rating           String   @db.VarChar(1)  // "A" | "B" | "C" | "X"

  unitNo      String   @db.VarChar(20) @map("unit_no")
  projectCode String   @db.VarChar(10) @map("project_code")
  snapshotAt  DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  commod    Commod              @relation(fields: [idMod], references: [idMod])
  equipment FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])
  creator   User?               @relation("InspectionCreatedBy", fields: [createdBy],
references: [idUser])

  @@index([fleetEquipmentId, idMod, type])
  @@index([insDate])
  @@index([projectCode])
  @@map("inspection")
}

model Condition {
  idCondition      Int    @id @default(autoincrement()) @map("id_condition")
  fleetEquipmentId Int    @map("id_unit")
  idMod            Int    @map("id_mod")
  condition        String @db.VarChar(20) // overall: CRITICAL | MONITOR | GOOD | NORMAL

  // Rating per sumber — sesuai logic agregasi legacy
  sosRating String? @db.VarChar(5) @map("sos_rating")
  fcRating  String? @db.VarChar(1) @map("fc_rating")
  mpsRating String? @db.VarChar(1) @map("mps_rating")
  viRating  String? @db.VarChar(1) @map("vi_rating")
  ta2Rating String? @db.VarChar(1) @map("ta2_rating")
  edRating  String? @db.VarChar(1) @map("ed_rating")

  evaluatedAt DateTime @default(now()) @map("evaluated_at")

  unitNo      String   @db.VarChar(20) @map("unit_no")
  projectCode String   @db.VarChar(10) @map("project_code")
  snapshotAt  DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  deletedAt DateTime? @map("deleted_at")

  commod    Commod              @relation(fields: [idMod], references: [idMod])
  equipment FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])

  @@unique([fleetEquipmentId, idMod])
  @@index([projectCode])
  @@index([condition])
  @@map("condition")
}

// ═══════════════════════════════════════════════════
```

```
// CANNIBALIZATION & APPROVAL
// ═══════════════════════════════════════════════════════

model Ba {
  idBa        Int      @id @default(autoincrement()) @map("id_ba")
  noBa        String   @unique @db.VarChar(20) @map("no_ba")
  projectCode String   @db.VarChar(10) @map("project_code")
  postingDate DateTime @db.Date @map("posting_date")
  symptom     String   @db.Text
  failure     String   @db.Text
  idCaused    Int      @map("id_caused")
  causedOther String   @default("") @db.VarChar(100) @map("caused_other")
  idStatus    Int      @map("id_status")
  statusOther String   @default("") @db.VarChar(100) @map("status_other")
  idAction    Int      @map("id_action")
  mrNo        String?  @db.VarChar(30) @map("mr_no")
  prNo        String?  @db.VarChar(30) @map("pr_no")
  poNo        String?  @db.VarChar(30) @map("po_no")
  statusBa    String   @default("OPEN") @db.VarChar(10) @map("status_ba")

  // Kolom flat legacy — tetap diisi untuk kompatibilitas export/laporan lama
  statusL1 String  @default("PENDING") @db.VarChar(15) @map("status_l1")
  userL1   String? @db.VarChar(100) @map("user_l1")
  statusL2 String  @default("PENDING") @db.VarChar(15) @map("status_l2")
  userL2   String? @db.VarChar(100) @map("user_l2")
  statusL3 String  @default("PENDING") @db.VarChar(15) @map("status_l3")
  userL3   String? @db.VarChar(100) @map("user_l3")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  createdBy Int?      @map("created_by")
  deletedAt DateTime? @map("deleted_at")

  kanibals          Kanibal[]
  approvals         BaApproval[]
  baCaused          BaCaused          @relation(fields: [idCaused], references:
[idCaused])
  baAction          BaAction          @relation(fields: [idAction], references:
[idAction])
  baComponentStatus BaComponentStatus @relation(fields: [idStatus], references:
[idStatus])
  creator           User?             @relation("BaCreatedBy", fields: [createdBy],
references: [idUser])

  @@index([projectCode])
  @@index([statusBa])
  @@index([statusL1, statusL2, statusL3, projectCode])
  @@map("ba")
}

/// Approval normalisasi — app baru tulis ke sini, sync ke kolom flat `ba.status_l*`
model BaApproval {
  idBaApproval Int       @id @default(autoincrement()) @map("id_ba_approval")
  idBa         Int       @map("id_ba")
  level        String    @db.VarChar(2)  // "L1" | "L2" | "L3"
  status       String    @default("PENDING") @db.VarChar(15) // PENDING | APPROVED | NOT
APPROVED
  approvedBy   Int?      @map("approved_by")
  approvedAt   DateTime? @map("approved_at")
```

```
  remark       String?   @db.Text

  ba   Ba    @relation(fields: [idBa], references: [idBa])
  user User? @relation(fields: [approvedBy], references: [idUser])

  @@unique([idBa, level])
  @@index([level, status])
  @@map("ba_approval")
}

model Kanibal {
  idKanibal        Int      @id @default(autoincrement()) @map("id_kanibal")
  noBa             String   @db.VarChar(20) @map("no_ba")
  idRep            Int?     @map("id_rep") // NULL = tidak ada WO (legacy: 0 → NULL)
  fleetEquipmentId Int      @map("id_unit")
  date             DateTime @db.Date
  compDesc         String   @db.VarChar(100) @map("comp_desc")
  pn               String   @default("") @db.VarChar(100)
  sn               String   @default("") @db.VarChar(100)
  pos              String   @default("") @db.VarChar(100)
  hmComp           Int      @default(0) @map("hm_comp")
  woNoKanibal      String?  @db.VarChar(30) @map("wo_no_kanibal")
  woStatusKanibal  String   @default("OPEN") @db.VarChar(100) @map("wo_status_kanibal")
  type             String   @db.VarChar(10) // "REMOVE" | "INSTALL"

  unitNo     String   @db.VarChar(20) @map("unit_no")
  snapshotAt DateTime @default(now())  @map("snapshot_at")

  createdAt DateTime  @default(now()) @map("created_at")
  updatedAt DateTime  @updatedAt      @map("updated_at")
  deletedAt DateTime? @map("deleted_at")

  ba          Ba                  @relation(fields: [noBa], references: [noBa])
  replacement Replacement?        @relation(fields: [idRep], references: [idRep])
  equipment   FleetEquipmentCache @relation(fields: [fleetEquipmentId], references:
[fleetEquipmentId])

  @@index([noBa, type])
  @@index([fleetEquipmentId])
  @@map("kanibal")
}

// ═══════════════════════════════════════════════════════
// LOOKUP TABLES
// ═══════════════════════════════════════════════════════

model BaCaused {
  idCaused Int    @id @default(autoincrement()) @map("id_caused")
  caused   String @db.VarChar(100)
  baDocuments Ba[]
  @@map("ba_caused")
}

model BaAction {
  idAction Int    @id @default(autoincrement()) @map("id_action")
  action   String @db.VarChar(100)
  baDocuments Ba[]
  @@map("ba_action")
}
```

```
model BaComponentStatus {
  idStatus Int    @id @default(autoincrement()) @map("id_status")
  status   String @db.VarChar(100)
  baDocuments Ba[]
  @@map("ba_status")
}

// ═══════════════════════════════════════════════════
// USER & AUTH
// ═══════════════════════════════════════════════════

model User {
  idUser      Int       @id @default(autoincrement()) @map("id_user")
  username    String    @unique @db.VarChar(50)
  password    String    @db.VarChar(255)
  fullName    String?   @db.VarChar(100) @map("full_name")
  level       String    @default("User") @db.VarChar(20)
  projectCode String?   @db.VarChar(10)  @map("project_code")
  sign        String?   @db.VarChar(5)   // Kanibal BA: "L1" | "L2" | "L3" | NULL
  pcrSign     String?   @db.VarChar(5)   @map("pcr_sign")
  // PCR Forecast BA: "PF" | "PS" | "PM" | "PLM" | "OD" | "FD" | "PD" | NULL
  // PF = Planner Foreman (pengaju) | PS = Plant Supervisor | PM = Project Manager
  // PLM = Plant Manager | OD/FD/PD = Direksi
  isActive    Boolean   @default(true)   @map("is_active")
  lastLogin   DateTime? @map("last_login")

  createdAt DateTime @default(now()) @map("created_at")
  updatedAt DateTime @updatedAt      @map("updated_at")

  baApprovals          BaApproval[]
  pcrForecastApprovals PcrForecastApproval[] @relation("PcrForecastApprover")
  hmCreated            Hm[]          @relation("HmCreatedBy")
  replacementsCreated  Replacement[] @relation("ReplacementCreatedBy")
  pcrForecastsCreated  PcrForecast[] @relation("PcrForecastCreatedBy")
  pcrForecastsSubmitted PcrForecast[] @relation("PcrForecastSubmittedBy")
  sosCreated           Sos[]         @relation("SosCreatedBy")
  inspectionsCreated   Inspection[]  @relation("InspectionCreatedBy")
  baCreated            Ba[]          @relation("BaCreatedBy")

  @@index([projectCode, sign])
  @@index([projectCode, pcrSign])
  @@map("user")
}
```

Catatan Penting Terkait Skema v2

Kolom Legacy	Perubahan v2	Alasan
`unit.id_unit`	`fleet_equipment_id` + FK ke `fleet_equipment_cache`	Jangkar lokal, join tanpa hit Fleet API
`unit.id_project` / `ba.id_project` / `user.id_project`	Kolom DB: `project_code` VARCHAR	Rename jelas, bukan INT dipakai string
`model.id_model` di `commod`	`id_model` = `fleet_model_id` dari Fleet API	Bukan FK lokal

Kolom Legacy	Perubahan v2	Alasan
`kanibal.id_rep = 0`	`id_rep = NULL`	FK valid ke `replacement`
`hm_unit`, `hm_rep`, `comp_life`, `life_percent`	`DECIMAL`	Hindari error float
`wo_no`, `mr_no`, `pr_no`, `po_no`	`VARCHAR(30)`	Support format dokumen panjang
`oil_change`, `oil_added`	`BOOLEAN`	Semantik benar (legacy 0/1)
`remarks` di replacement	`TEXT`	Tidak terpotong 50 char
`sos.4um` dll	`p4um` + `@map("4um")`	Constraint Prisma
Denormalized `unit_no`/`project_code`	+ `snapshot_at`	Audit snapshot, tidak di-update retroaktif
Approval flat di `ba`	+ tabel `ba_approval`	Audit timestamp + FK user, kolom lama tetap
`condition`	+ 6 kolom rating per sumber	Logic agregasi legacy utuh
—	`legacy_unit_mapping`	Dokumentasi migrasi id_unit
—	`pcr_forecast`	Modul forecast PCR per periode/quarter
—	`pcr_forecast_approval`	6 level approval BA PCR (PS → PM/PLM → Direksi)
—	`user.pcr_sign`	Role PCR terpisah dari `sign` kanibal L1/L2/L3
—	`replacement.id_rep` ← `pcr_forecast`	Satu forecast → satu WO actual
—	`created_at`, `updated_at`, `deleted_at`	Audit trail
—	`@@unique([id_model, id_comp])` di commod	Cegah duplikat policy

Sync Fleet Equipment Cache

Jalankan periodik (cron / job saat startup):

```
// lib/fleet-api/sync-cache.ts
import { prisma } from "@/lib/prisma"
import { getEquipments } from "@/lib/fleet-api/client"

export async function syncFleetEquipmentCache() {
  const equipments = await getEquipments()
  for (const eq of equipments) {
    await prisma.fleetEquipmentCache.upsert({
      where: { fleetEquipmentId: eq.id },
      create: {
        fleetEquipmentId: eq.id,
```

```
        unitNo: eq.unit_no,
        description: eq.description,
        projectCode: eq.project_code,
        fleetModelId: eq.model_id,
        modelName: eq.model,
        manufacture: eq.manufacture,
        unitStatus: eq.unitstatus,
      },
      update: {
        unitNo: eq.unit_no,
        description: eq.description,
        projectCode: eq.project_code,
        fleetModelId: eq.model_id,
        modelName: eq.model,
        manufacture: eq.manufacture,
        unitStatus: eq.unitstatus,
        syncedAt: new Date(),
      },
    })
  }
}
```

Sync Approval Flat ↔ BaApproval

Saat approve/reject di app baru, tulis ke `ba_approval` **dan** update kolom flat legacy:

```
// actions/approvals.ts (contoh)
async function approveBa(idBa: number, level: "L1" | "L2" | "L3", userId: number) {
  const now = new Date()
  await prisma.$transaction([
    prisma.baApproval.upsert({
      where: { idBa_level: { idBa, level } },
      create: { idBa, level, status: "APPROVED", approvedBy: userId, approvedAt: now },
      update: { status: "APPROVED", approvedBy: userId, approvedAt: now },
    }),
    prisma.ba.update({
      where: { idBa },
      data: level === "L1" ? { statusL1: "APPROVED", userL1: String(userId) }
           : level === "L2" ? { statusL2: "APPROVED", userL2: String(userId) }
           : { statusL3: "APPROVED", userL3: String(userId) },
    }),
  ])
}
```

6.1 Modul PCR Forecast — Domain & Alur

Modul ini mengakomodasi data perencanaan PCR seperti spreadsheet operasional. **BA PCR di forecast bukan sama dengan tabel `ba` (kanibal)** — itu dokumen persetujuan formal untuk rencana ganti komponen dengan alur approval organisasi berikut.

Alur Bisnis Umum (End-to-End)

```
Forecasting
    ↓
Pengajuan PCR + BA PCR  ←  Planner Foreman
    ↓
Pemeriksaan             ←  Plant Supervisor
    ↓
Persetujuan Manajemen   ←  Project Manager  +  Plant Manager  (keduanya wajib)
    ↓
Tinjauan Direksi        ←  Operation Director + Finance Director + President Director
(ketiganya wajib)
    ↓
Realisasi PCR           ←  Record di `replacement` (modul PCR actual / aplikasi lama)
```

Tahap	Aktivitas	Pelaku	Aksi di Sistem
1	**Forecasting**	Planner / Admin	Buat baris forecast, snapshot HM/life/SOS, set `plan_period` & `quarter`
2	**Pengajuan BA PCR**	**Planner Foreman** (`pcr_sign=PF`)	Submit BA → seed 6 baris `pcr_forecast_approval`, set `ba_submitted_at`
3	**Pemeriksaan**	**Plant Supervisor** (`pcr_sign=PS`)	Approve/reject tahap 1 — verifikasi data teknis rencana
4	**Persetujuan Manajemen**	**Project Manager** (`PM`) + **Plant Manager** (`PLM`)	Approve paralel — keduanya harus APPROVED sebelum lanjut
5	**Tinjauan Direksi**	**Operation Director** (`OD`), **Finance Director** (`FD`), **President Director** (`PD`)	Approve paralel — ketiganya harus APPROVED
6	**Realisasi PCR**	Field team / Admin	Convert → buat `replacement` WO → close WO dengan `action_date`, PO, remark

> **Catatan:** `sign` (L1/L2/L3) hanya untuk **BA Kanibal**. Approval BA PCR memakai field terpisah `user.pcr_sign`.

Role & Mapping `user.pcr_sign`

Kode	Role	Tahap	Urutan	Paralel dengan
`PF`	Planner Foreman	Pengaju	—	—
`PS`	Plant Supervisor	Pemeriksaan	1	—
`PM`	Project Manager	Persetujuan manajemen	2	`PLM`
`PLM`	Plant Manager	Persetujuan manajemen	2	`PM`
`OD`	Operation Director	Tinjauan direksi	3	`FD`, `PD`
`FD`	Finance Director	Tinjauan direksi	3	`OD`, `PD`
`PD`	President Director	Tinjauan direksi	3	`OD`, `FD`

Saat submit BA PCR, sistem otomatis membuat 6 record `pcr_forecast_approval` (PS, PM, PLM, OD, FD, PD) dengan `status=PENDING`.

Pemetaan Kolom Spreadsheet → Database

Kolom Spreadsheet	Field `pcr_forecast`	Sumber / Catatan
Model Unit	`model_name`	Snapshot dari `fleet_equipment_cache.model_name`
No Unit	`unit_no` + FK `id_unit`	Fleet API / cache
Component	`comp_desc` + FK `id_mod`	Join `commod` → `comp`
HM Component	`hm_component`	HM terbaru + last replacement + `comp_hour`
Policy	`policy`	`commod.policy`
Life Time Component	`life_percent`	Kalkulasi `lib/calculations/life.ts`
Rating S.O.S	`rating_sos`	`condition.rating_sos` atau SOS terbaru
Price Component	`price_component`	`commod.price`
Plan Periode	`plan_period`	Input/import (May-26 → `2026-05-01`)
Site	`project_code`	Fleet equipment / filter user
Quarter	`quarter`	Q1–Q4 (derived dari `plan_period` atau input)
BA PCR	`ba_pcr_status`	PENDING → SUBMITTED → IN_REVIEW → APPROVED (= DONE di spreadsheet)
Status BA PCR	`status_ba_pcr`	Auto: "Wait Plant Spv", "Wait Project Manager", "Wait Direksi", dll.
Tanggal Pengajuan BA PCR	`ba_submitted_at` + `submitted_by`	Set saat Planner Foreman submit
STATUS PCR	`status`	OPEN (rencana/ongoing) / CLOSED (realisasi selesai)
ACTION DATE PCR	`action_date`	Tanggal eksekusi actual
PO	`po_number`	Nomor PO saat close
REMARK	`remark`	Catatan bebas

Diagram Workflow

```
flowchart TD
  A["1. Forecasting
(snapshot HM, life %, SOS, plan periode)"]
  B["2. Pengajuan BA PCR
Planner Foreman (PF)"]
  C["3. Pemeriksaan
Plant Supervisor (PS)"]
  D["4. Persetujuan Manajemen"]
  D1["Project Manager (PM)"]
  D2["Plant Manager (PLM)"]
  E["5. Tinjauan Direksi"]
  E1["Operation Director (OD)"]
  E2["Finance Director (FD)"]
```

```
    E3["President Director (PD)"]
    F["6. Realisasi PCR
  Convert → replacement WO"]
    G["Close WO
  action_date + PO + remark"]

    A --> B
    B --> C
    C -->|Approved| D
    C -->|Rejected| A
    D --> D1
    D --> D2
    D1 --> E
    D2 --> E
    E --> E1
    E --> E2
    E --> E3
    E1 --> F
    E2 --> F
    E3 --> F
    F --> G
```

Gate antar tahap (validasi app):

- Tahap 2→3: `PS` harus `APPROVED` sebelum `PM`/`PLM` bisa action
- Tahap 4→5: `PM` **dan** `PLM` keduanya `APPROVED` sebelum `OD`/`FD`/`PD` bisa action
- Tahap 5→6: `OD`, `FD`, **dan** `PD` ketiganya `APPROVED` → `ba_pcr_status=APPROVED` (spreadsheet: BA PCR = DONE)
- Tahap 6: convert hanya jika `ba_pcr_status=APPROVED`

Aturan bisnis lainnya:

1. Satu baris forecast aktif (`status=OPEN`) per `(fleet_equipment_id, id_mod)` — validasi di app layer.
2. **Refresh snapshot**: update `hm_component`, `life_percent`, `rating_sos` tanpa reset approval yang sudah jalan (hanya jika `ba_pcr_status=PENDING`).
3. **Reject** di tahap manapun: set `ba_pcr_status=REJECTED`, forecast kembali ke Planner Foreman untuk revisi & re-submit.
4. Saat convert: buat `replacement` dengan `wo_status=OPEN`; set `pcr_forecast.id_rep`.
5. Saat close WO (flow existing): sync forecast `status=CLOSED`, `action_date`, `po_number`.
6. Import Excel historical: baris `STATUS PCR=Closed` → buat forecast + replacement sekaligus.

Generate Forecast Otomatis (opsional)

Selain input manual/import Excel, app bisa generate baris forecast dari:

- Equipment aktif di project (Fleet cache)
- Semua `commod` untuk `fleet_model_id` unit tersebut
- `life_percent` ≥ threshold (mis. 70%) atau `rating_sos` ∈ {C, X}
- `plan_period` = hasil `calculateForecast()` dari HM 3 bulan terakhir

Endpoint: `POST /api/forecasts/generate?projectCode=&quarter=&year=`

Inisialisasi Approval saat Submit BA PCR

```ts
// lib/forecasts/approval-workflow.ts
export const PCR_APPROVAL_LEVELS = [
  { level: "PS",  stepOrder: 1, label: "Plant Supervisor" },
  { level: "PM",  stepOrder: 2, label: "Project Manager" },
  { level: "PLM", stepOrder: 2, label: "Plant Manager" },
  { level: "OD",  stepOrder: 3, label: "Operation Director" },
  { level: "FD",  stepOrder: 3, label: "Finance Director" },
  { level: "PD",  stepOrder: 3, label: "President Director" },
] as const;

export function getPendingApprovalLabel(
  approvals: Array<{ level: string; status: string }>,
): string {
  const pending = approvals.find((a) => a.status === "PENDING");
  if (!pending) return "BA PCR Approved";
  const map: Record<string, string> = {
    PS: "Wait Plant Supervisor",
    PM: "Wait Project Manager",
    PLM: "Wait Plant Manager",
    OD: "Wait Operation Director",
    FD: "Wait Finance Director",
    PD: "Wait President Director",
  };
  return map[pending.level] ?? "Wait Approval";
}

export function canApproveAtStep(
  approvals: Array<{ level: string; stepOrder: number; status: string }>,
  targetLevel: string,
): boolean {
  const target = approvals.find((a) => a.level === targetLevel);
  if (!target || target.status !== "PENDING") return false;

  // Tahap 1: PS — langsung bisa jika SUBMITTED
  if (target.stepOrder === 1) return true;

  // Tahap 2: PM/PLM — PS harus sudah APPROVED
  if (target.stepOrder === 2)
    return approvals.find((a) => a.level === "PS")?.status === "APPROVED";

  // Tahap 3: Direksi — PM dan PLM harus APPROVED
  if (target.stepOrder === 3) {
    const pm = approvals.find((a) => a.level === "PM");
    const plm = approvals.find((a) => a.level === "PLM");
    return pm?.status === "APPROVED" && plm?.status === "APPROVED";
  }
  return false;
}
```

---

7. Pemetaan Fitur: Legacy → Modern

Autentikasi

Legacy (CI)	Modern (Next.js)

Legacy (CI)	Modern (Next.js)
CI Session (`isLogin`, `level`, `kode_project`)	NextAuth.js JWT session
Password plaintext di DB	bcrypt hash (salt rounds 12)
Redirect manual `redirect('login/process_login')`	Middleware Next.js
Bug: `$level` undefined saat login	Zod validation + typed session
Project scope via `kode_project` di session	`projectCode` di JWT token

Master Data

Modul	Legacy	Modern
Project	`project.php` — CRUD lokal	**Read-only dari Fleet API** (dropdown, filter)
Model	`model.php` — CRUD lokal	**Read-only dari Fleet API** (model_id dari equipments)
Unit/Equipment	`unit.php` — CRUD + import	**Read-only dari Fleet API** (tidak ada CRUD lokal)
Component	`component.php` — CRUD + import	`/components` — CRUD lokal tetap ada
Model-Component	`model.php` detail	`/model-components` — CRUD lokal (policy mapping)
User	`user.php` — CRUD	`/users` (admin only) — lokal

Operasional

Modul	Legacy	Modern Route
Hour Meter	`hm.php`	`/hour-meters` (input HM untuk equipment dari fleet)
PCR Forecast	*(baru — spreadsheet)*	`/forecasts`, `/forecasts/[id]`
PCR/Replacement	`unit.php`	`/equipments/[fleetId]/replacements`
SOS	`unit.php`	`/equipments/[fleetId]/sos`
Filter Cut	`unit.php`	`/equipments/[fleetId]/inspections/filter-cut`
Magnetic Plug	`unit.php`	`/equipments/[fleetId]/inspections/magnetic`
Visual	`unit.php`	`/equipments/[fleetId]/inspections/visual`
TA2	`unit.php`	`/equipments/[fleetId]/inspections/ta2`
Electronic Data	`unit.php`	`/equipments/[fleetId]/inspections/electronic`
Condition	`unit.php`	`/equipments/[fleetId]/condition`

> **Catatan naming:** Route menggunakan `fleetId` (integer id dari Fleet API), bukan ID lokal.

Laporan & Export

Modul	Legacy	Modern
PCR Export	`pcr.php` → PHPExcel	`/api/exports/pcr` → ExcelJS
Forecast Export	*(spreadsheet)*	`/api/exports/forecasts`
SOS Export	`sos.php` → PHPExcel	`/api/exports/sos`

Modul	Legacy	Modern
Inspection Export	`inspection.php` → PHPExcel	`/api/exports/inspections`
Condition Export	`condition.php` → PHPExcel	`/api/exports/conditions`
Cannibal Export	`cannibal.php` → PHPExcel	`/api/exports/cannibal`
Import SOS/HM	`unit.php` upload	`/api/imports/[type]`
Import Forecast	*(spreadsheet)*	`/api/imports/forecasts`

> **Dihapus:** Import Excel untuk Unit (tidak relevan — unit dari Fleet API)

---

8. Struktur Folder Proyek

Sesuaikan dengan struktur yang sudah ada di Vuexy template. Tambahkan file berikut:

```
(root proyek Vuexy)/
│
├── src/                          # atau app/ — sesuai struktur Vuexy
│   ├── app/                      # Next.js App Router
│   │   ├── (auth)/
│   │   │   └── login/page.tsx
│   │   ├── (dashboard)/
│   │   │   ├── layout.tsx
│   │   │   ├── page.tsx                    # Dashboard
│   │   │   │
│   │   │   ├── equipments/                 # List unit dari Fleet API
│   │   │   │   ├── page.tsx
│   │   │   │   └── [fleetId]/
│   │   │   │       ├── page.tsx            # Detail unit + ringkasan komponen
│   │   │   │       ├── replacements/
│   │   │   │       │   ├── page.tsx
│   │   │   │       │   └── [id]/page.tsx
│   │   │   │       ├── sos/
│   │   │   │       │   ├── page.tsx
│   │   │   │       │   └── [id]/page.tsx
│   │   │   │       ├── inspections/
│   │   │   │       │   ├── [type]/
│   │   │   │       │   │   ├── page.tsx
│   │   │   │       │   │   └── [id]/page.tsx
│   │   │   │       └── condition/
│   │   │   │           └── page.tsx
│   │   │   │
│   │   │   ├── hour-meters/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   │
│   │   │   ├── forecasts/                  # PCR Forecast (NEW)
│   │   │   │   ├── page.tsx                # Grid filter project/quarter/status
│   │   │   │   ├── import/page.tsx         # Upload Excel spreadsheet
│   │   │   │   └── [id]/
│   │   │   │       ├── page.tsx            # Detail + BA PCR + convert
│   │   │   │       └── approvals/page.tsx  # Antrean approval BA PCR
│   │   │   │
│   │   │   ├── components/                 # Katalog komponen PCR
```

```
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   │
│   │   │   ├── model-components/          # Policy mapping
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   │
│   │   │   ├── cannibal/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/
│   │   │   │       ├── page.tsx
│   │   │   │       └── print/page.tsx
│   │   │   │
│   │   │   ├── approvals/
│   │   │   │   ├── page.tsx
│   │   │   │   └── [id]/page.tsx
│   │   │   │
│   │   │   ├── reports/
│   │   │   │   ├── pcr/page.tsx
│   │   │   │   ├── forecasts/page.tsx     # Summary forecast per quarter
│   │   │   │   ├── sos/page.tsx
│   │   │   │   ├── inspections/page.tsx
│   │   │   │   └── conditions/page.tsx
│   │   │   │
│   │   │   └── users/                     # Admin only
│   │   │       ├── page.tsx
│   │   │       └── [id]/page.tsx
│   │   │
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts
│   │       │
│   │       ├── fleet/                     # Proxy ke Fleet API (untuk client components)
│   │       │   ├── projects/route.ts
│   │       │   └── equipments/route.ts
│   │       │
│   │       ├── components/route.ts
│   │       ├── model-components/route.ts
│   │       ├── hour-meters/route.ts
│   │       ├── forecasts/                 # PCR Forecast (NEW)
│   │       │   ├── route.ts
│   │       │   ├── generate/route.ts      # Auto-generate dari life/SOS
│   │       │   └── [id]/
│   │       │       ├── route.ts
│   │       │       ├── refresh/route.ts   # Refresh snapshot metrics
│   │       │       ├── submit-ba/route.ts
│   │       │       ├── convert/route.ts   # → buat replacement WO
│   │       │       └── close/route.ts     # Close tanpa WO (historical import)
│   │       ├── forecast-approvals/        # BA PCR approval queue
│   │       │   └── [id]/
│   │       │       ├── approve/route.ts
│   │       │       └── reject/route.ts
│   │       ├── replacements/
│   │       │   ├── route.ts
│   │       │   └── [id]/
│   │       │       ├── route.ts
│   │       │       ├── close/route.ts
│   │       │       └── report/route.ts
│   │       ├── sos/route.ts
│   │       ├── inspections/route.ts
```

```
│   │       ├── cannibal/
│   │       │   ├── route.ts
│   │       │   └── [id]/
│   │       │       ├── route.ts
│   │       │       └── submit/route.ts
│   │       ├── approvals/
│   │       │   └── [id]/
│   │       │       ├── approve/route.ts
│   │       │       └── reject/route.ts
│   │       ├── exports/
│   │       │   ├── pcr/route.ts
│   │       │   ├── forecasts/route.ts
│   │       │   ├── sos/route.ts
│   │       │   ├── inspections/route.ts
│   │       │   ├── conditions/route.ts
│   │       │   └── cannibal/route.ts
│   │       └── imports/
│   │           ├── forecasts/route.ts
│   │           ├── sos/route.ts
│   │           └── hour-meters/route.ts
│
├── lib/
│   ├── prisma.ts                          # Prisma client singleton
│   ├── auth.ts                            # NextAuth config
│   ├── fleet-api/
│   │   ├── client.ts                      # Fleet API fetcher + cache
│   │   └── sync-cache.ts                  # Sync ke fleet_equipment_cache
│   ├── calculations/
│   │   ├── life.ts                        # % life komponen
│   │   └── forecast.ts                    # Forecast hari ganti
│   ├── forecasts/
│   │   ├── approval-workflow.ts           # Gate tahap PS→PM/PLM→Direksi
│   │   └── build-snapshot.ts              # Snapshot HM/life/SOS saat forecast
│   ├── excel/
│   │   ├── exporters/
│   │   │   ├── pcr.ts
│   │   │   ├── forecasts.ts               # Export layout spreadsheet
│   │   │   ├── sos.ts
│   │   │   ├── inspection.ts
│   │   │   └── cannibal.ts
│   │   └── importers/
│   │       ├── forecasts.ts               # Import Excel spreadsheet
│   │       ├── sos.ts
│   │       └── hour-meters.ts
│   ├── validations/
│   │   ├── forecast.ts
│   │   ├── replacement.ts
│   │   ├── sos.ts
│   │   ├── inspection.ts
│   │   └── cannibal.ts
│   └── utils/
│       ├── project-scope.ts
│       └── rating-color.ts                # A/B/C/X → warna
│
├── actions/                               # Server Actions
│   ├── forecasts.ts
│   ├── replacements.ts
│   ├── sos.ts
│   ├── inspections.ts
```

```
│   ├── cannibal.ts
│   └── approvals.ts
│
├── types/
│   ├── fleet-api.ts                    # FleetProject, FleetEquipment
│   ├── forecast.ts                     # PcrSign, PcrApprovalLevel, Excel row type
│   ├── next-auth.d.ts                  # Extend session type
│   └── index.ts
│
├── scripts/
│   ├── migrate-passwords.ts
│   ├── map-units-to-fleet.ts           # legacy_unit_mapping
│   └── seed-ba-approval.ts             # ba_approval dari kolom flat legacy
│
├── prisma/
│   ├── schema.prisma
│   ├── migrations/
│   └── seed.ts
│
├── middleware.ts
├── .env.local
└── next.config.ts
```

---

9. API Routes Mapping

Fleet Proxy (server ke Fleet API)

```
GET  /api/fleet/projects          → proxy ke 192.168.32.15/ark-fleet/api/projects
GET  /api/fleet/equipments        → proxy ke /equipments (difilter by project user)
GET  /api/fleet/equipments/:id    → proxy ke /equipments/:id
```

Components (lokal)

```
GET    /api/components            → Daftar komponen
POST   /api/components            → Buat komponen baru
GET    /api/components/:id        → Detail
PUT    /api/components/:id        → Update
DELETE /api/components/:id        → Hapus
```

Model-Components / Policy Mapping (lokal)

```
GET    /api/model-components?fleetModelId=   → List policy per model
POST   /api/model-components                 → Tambah mapping baru
PUT    /api/model-components/:id             → Update policy/price
DELETE /api/model-components/:id             → Hapus
```

Hour Meters

```
GET    /api/hour-meters?fleetEquipmentId=    → Riwayat HM satu unit
GET    /api/hour-meters?projectCode=         → HM per project
POST   /api/hour-meters                      → Input HM baru
PUT    /api/hour-meters/:id                  → Edit
DELETE /api/hour-meters/:id                  → Hapus
```

PCR Forecast (NEW)

```
GET    /api/forecasts?projectCode=&quarter=&year=&status=&baPcrStatus=
POST   /api/forecasts                        → Buat forecast (manual)
POST   /api/forecasts/generate               → Generate otomatis dari life/SOS threshold
GET    /api/forecasts/:id
PUT    /api/forecasts/:id                    → Edit plan_period, remark, dll.
DELETE /api/forecasts/:id                    → Soft delete
POST   /api/forecasts/:id/refresh            → Refresh snapshot HM/life/rating/price
POST   /api/forecasts/:id/submit-ba          → Ajukan BA PCR (Planner Foreman; seed 6
approval rows)
POST   /api/forecasts/:id/convert            → Convert → buat `replacement` WO (setelah
semua approve)
POST   /api/forecasts/:id/close              → Close forecast (historical / tanpa WO)
```

Forecast Approvals (BA PCR — role organisasi)

```
GET    /api/forecast-approvals?pcrSign=PS|PM|PLM|OD|FD|PD  → Antrean per role user
POST   /api/forecast-approvals/:id/approve                 → Approve (validasi gate tahap)
POST   /api/forecast-approvals/:id/reject                  → Reject → kembali ke Planner
Foreman
```

> Queue filter by `session.user.pcrSign`. User `pcr_sign=PF` melihat forecast yang dia ajukan; approver hanya melihat baris yang gate tahap sebelumnya sudah terpenuhi.

Replacements / PCR

```
GET    /api/replacements?fleetEquipmentId=&modelComponentId=&status=
POST   /api/replacements
GET    /api/replacements/:id
PUT    /api/replacements/:id
DELETE /api/replacements/:id
POST   /api/replacements/:id/close      → Tutup WO + hitung % life
POST   /api/replacements/:id/report     → Upload PDF laporan
DELETE /api/replacements/:id/report     → Hapus file laporan
```

SOS

```
GET    /api/sos?fleetEquipmentId=&modelComponentId=
POST   /api/sos
GET    /api/sos/:id
```

```
PUT    /api/sos/:id
DELETE /api/sos/:id
```

Inspections

```
GET    /api/inspections?fleetEquipmentId=&type=&rating=
POST   /api/inspections
GET    /api/inspections/:id
PUT    /api/inspections/:id
DELETE /api/inspections/:id
```

Cannibal / BA

```
GET    /api/cannibal?projectCode=&status=
POST   /api/cannibal
GET    /api/cannibal/:id
PUT    /api/cannibal/:id
DELETE /api/cannibal/:id
POST   /api/cannibal/:id/submit
POST   /api/cannibal/:id/cancel
POST   /api/cannibal/:id/close
```

Approvals

```
GET    /api/approvals?sign=L1|L2|L3          → Antrean approval
POST   /api/approvals/:id/approve            → Setujui
POST   /api/approvals/:id/reject             → Tolak
```

Exports

```
GET    /api/exports/pcr?projectCode=&fleetEquipmentId=&status=
GET    /api/exports/forecasts?projectCode=&quarter=&year=&status=
GET    /api/exports/sos?projectCode=&from=&to=
GET    /api/exports/inspections?type=&projectCode=
GET    /api/exports/conditions?projectCode=
GET    /api/exports/cannibal?projectCode=&status=
```

Imports

```
POST   /api/imports/hour-meters   → Upload Excel bulk HM
POST   /api/imports/forecasts     → Upload Excel PCR Forecast (layout spreadsheet)
POST   /api/imports/sos           → Upload Excel bulk SOS
```

---

10. Rencana Implementasi Bertahap

Phase 0: Persiapan (1-2 hari)

- [ ] Backup database produksi: `mysqldump arka_pcr > backup_$(date +%Y%m%d).sql`
- [ ] Jalankan `SHOW CREATE TABLE` untuk semua tabel aktual (dapatkan schema real)
- [ ] Khususnya `DESCRIBE sos` untuk mendapat 47+ field aktual
- [ ] Verifikasi konektivitas ke Fleet API: `curl http://192.168.32.15/ark-fleet/api/projects`
- [ ] Cek versi Next.js di proyek Vuexy yang ada: `cat package.json | grep next`
- [ ] Setup environment: Node.js 20 LTS, MySQL 8
- [ ] **Buat mapping unit:** export `unit` lama + cocokkan dengan `/ark-fleet/api/equipments` → isi `legacy_unit_mapping`
- [ ] **Dedupe `commod`:** cek duplikat `(id_model, id_comp)`, remap FK sebelum import
- [ ] **Transform kanibal:** `UPDATE kanibal SET id_rep = NULL WHERE id_rep = 0`

Phase 1: Pondasi di Proyek Vuexy yang Ada (2-3 hari)

- [ ] Install dependensi tambahan yang belum ada di Vuexy:

```
# Cek dulu yang sudah ada: cat package.json
npm install prisma @prisma/client
npm install next-auth@beta bcryptjs zod react-hook-form @hookform/resolvers
npm install exceljs axios
npm install -D @types/bcryptjs
```

- [ ] Buat `prisma/schema.prisma` (**v2** dari Section 6 dokumen ini)
- [ ] Jalankan `npx prisma migrate dev --name init_v2`
- [ ] Buat `lib/prisma.ts` (singleton client)
- [ ] Setup NextAuth.js: `lib/auth.ts` + route `app/api/auth/[...nextauth]/route.ts`
- [ ] Extend session types di `types/next-auth.d.ts`
- [ ] Setup `middleware.ts` untuk proteksi route
- [ ] Buat `lib/fleet-api/client.ts` + **`lib/fleet-api/sync-cache.ts`**
- [ ] Jalankan sync pertama: populate `fleet_equipment_cache` dari Fleet API
- [ ] Test koneksi Fleet API dari server Next.js
- [ ] Buat `types/fleet-api.ts`

Phase 2: Fleet API Integration & Master Data PCR (3-4 hari)

- [ ] Buat proxy route `/api/fleet/projects` dan `/api/fleet/equipments`
- [ ] Implementasi `lib/utils/project-scope.ts` (filter `000H`)
- [ ] Setup cron/job sync `fleet_equipment_cache` (interval 10 menit)
- [ ] Modul **Component**: list, create, edit, delete, import Excel
- [ ] Modul **Model-Component Policy**:
  - List policy per fleet model (dropdown model dari Fleet API)
  - Tambah/edit/hapus policy komponen
  - Enforce unique `(fleetModelId, idComp)`
- [ ] Modul **User** (admin only): list, create, edit, delete + hash password + assign `sign` (kanibal) dan `pcr_sign` (forecast)
- [ ] Script `scripts/migrate-passwords.ts` untuk hash password lama
- [ ] Migrasi data `comp` dan `commod` dari DB lama (dedupe + remap `id_model` ke Fleet API)

Phase 3: Equipment (Unit) — Read dari Fleet API (2-3 hari)

- [ ] Halaman `/equipments`: daftar unit dari Fleet API (dengan filter project, status, search)

- [ ] Gunakan komponen tabel Vuexy yang sudah ada
- [ ] Filter project sesuai scope user (`projectCode`)
- [ ] Badge status: ACTIVE (hijau) / IN-ACTIVE (merah)
- [ ] Halaman `/equipments/[fleetId]`: detail unit + ringkasan komponen yang punya data PCR
- [ ] Tidak ada form create/edit/delete unit — semua read-only dari Fleet API

Phase 4: Hour Meter (2-3 hari)

- [ ] Halaman `/hour-meters`: list HM dengan filter project + equipment
- [ ] Form tambah HM (pilih equipment dari Fleet API dropdown)
- [ ] Edit & delete HM
- [ ] Import bulk HM via Excel
- [ ] Export template HM
- [ ] Migrasi data `hm` dari DB lama (map `id_unit` lama → `fleetEquipmentId`)

Phase 5: PCR Forecast (5-6 hari) — NEW

- [ ] Grid forecast: filter `projectCode`, `quarter`, `plan_period`, `status`, `ba_pcr_status`
- [ ] Color-code `life_percent` (≥100% merah) dan `rating_sos` (A/B/C/X) sesuai spreadsheet
- [ ] Create forecast manual: pilih equipment (Fleet) + komponen (`id_mod`)
- [ ] Auto-populate snapshot: HM, policy, life %, rating SOS, price dari HM/commod/condition
- [ ] **Refresh metrics** per baris / bulk refresh per quarter
- [ ] **Generate forecast** otomatis dari threshold life % / rating SOS
- [ ] Submit BA PCR oleh **Planner Foreman** (`pcr_sign=PF`) → seed approval PS/PM/PLM/OD/FD/PD
- [ ] Antrean approval per role: PS → PM+PLM (paralel) → OD+FD+PD (paralel)
- [ ] Kolom `status_ba_pcr` auto-sync ("Wait Plant Supervisor", "Wait Project Manager", dll.)
- [ ] **Convert to Actual** setelah semua direksi approve → buat `replacement` + link `id_rep`
- [ ] Close forecast: set `action_date`, `po_number`, `status=CLOSED` (sync dari close WO atau input langsung)
- [ ] Import Excel layout spreadsheet (18 kolom) — mode `OPEN` dan mode `CLOSED` historical
- [ ] Export Excel dengan format yang sama dengan spreadsheet operasional
- [ ] Validasi: max 1 forecast OPEN per `(equipment, komponen)`

Phase 6: PCR / Replacement (4-5 hari)

- [ ] List WO per equipment per komponen (route `/equipments/[fleetId]/replacements`)
- [ ] Create WO baru (pilih `idMod` dari mapping commod untuk fleet model)
- [ ] Edit WO + upload file laporan (PDF)
- [ ] Close WO: simpan `comp_life`/`life_percent` ke DB (snapshot legacy) + set `life_calculated_at`
- [ ] **UI % life dihitung on-the-fly** dari HM terbaru (tidak hanya dari kolom stored)
- [ ] Visualisasi life progress per komponen (progress bar dari Vuexy)
- [ ] Export PCR Excel per komponen dan summary
- [ ] Migrasi data `replacement` (map `id_unit` → fleet id via `legacy_unit_mapping`)

Phase 7: SOS (4-5 hari)

- [ ] List SOS per equipment per komponen
- [ ] Form SOS dengan 47+ field (grouping: Wear Metals, Physical Properties, Evaluation)
- [ ] Color-coding evaluation code A/B/C/X
- [ ] Export SOS Excel dengan warna
- [ ] Import SOS bulk via Excel
- [ ] Migrasi data `sos`

Phase 8: Inspections & Condition (5-6 hari)

- [ ] 5 tipe inspeksi (masing-masing form + list): Filter Cut, Magnetic, Visual, TA2, Electronic Data
- [ ] Rating A/B/C/X dengan warna badge
- [ ] Export inspeksi per tipe
- [ ] Modul Condition: agregasi SOS + 5 inspeksi → simpan `condition` + 6 kolom rating per sumber
- [ ] Tampilan kondisi di halaman detail equipment (tabel per komponen)
- [ ] Migrasi data `inspection` dan `condition` (hitung ulang rating per sumber dari historis)

Phase 9: Cannibal & Approval (5-6 hari)

- [ ] Daftar BA dengan filter project + status
- [ ] Form buat BA: pilih equipment dari Fleet API, baris REMOVE/INSTALL dinamis
- [ ] BA series (cannibal + link ke PCR WO)
- [ ] Status workflow: Draft → Submitted → Approved/Rejected → Closed
- [ ] Modul **Approval**: tulis ke `ba_approval` + sync kolom flat `status_l1/l2/l3` di `ba`
- [ ] Filter queue berdasarkan `sign` user (L1/L2/L3)
- [ ] Filter berdasarkan `projectCode`
- [ ] Print view BA (halaman khusus tanpa layout)
- [ ] Export BA Excel
- [ ] Migrasi data `ba` → seed `ba_approval` dari kolom flat legacy
- [ ] Migrasi data `kanibal` (`id_rep`: 0 → NULL)

Phase 10: Dashboard & Laporan (3-4 hari)

- [ ] Dashboard: stat pending approval per level, cannibal belum approve, komponen kritis
- [ ] Widget komponen kritis (% life ≥ 85%)
- [ ] Widget forecast quarter: total OPEN vs CLOSED, nilai `price_component` agregat
- [ ] Halaman Summary Forecast (filter quarter + export)
- [ ] Halaman Summary PCR (filter + export)
- [ ] Halaman Summary SOS (filter + export)
- [ ] Halaman Summary Inspeksi (filter + export)
- [ ] Halaman Summary Condition (filter + export)

Phase 11: Polish & Deployment (3-4 hari)

- [ ] Loading states (skeleton sesuai style Vuexy)
- [ ] Error boundary dan pesan error yang informatif
- [ ] Validasi semua form dengan Zod
- [ ] Test kalkulasi life & forecast (`vitest`)
- [ ] Setup `.env` production
- [ ] Build: `npm run build`
- [ ] Deploy (PM2 + Nginx atau sesuai infrastruktur yang ada)

---

11. Detail Implementasi Per Modul

11.1 Kalkulasi Life Komponen

```
// lib/calculations/life.ts

export interface LifeCalcInput {
  hmNow: number; // HM unit terkini (dari tabel hour_meters)
  hmLastReplacement: number; // HM saat ganti terakhir (dari replacement.hmReplacement)
  compHour: number; // Jam komponen saat dipasang (replacement.compHour)
```

```
  policy: number; // Kebijakan jam ganti (modelComponent.policy)
}

export interface LifeCalcResult {
  currentLife: number;
  lifePercent: number;
  remainingHours: number;
  isCritical: boolean; // >= 85%
  isOverdue: boolean; // >= 100%
}

export function calculateComponentLife(input: LifeCalcInput): LifeCalcResult {
  const currentLife = input.hmNow - input.hmLastReplacement + input.compHour;
  const lifePercent = (currentLife / input.policy) * 100;
  const remainingHours = input.policy - currentLife;

  return {
    currentLife,
    lifePercent: Math.round(lifePercent * 100) / 100,
    remainingHours,
    isCritical: lifePercent >= 85,
    isOverdue: lifePercent >= 100,
  };
}

export function calculateForecast(
  remainingHours: number,
  hmReadings: Array<{ readingDate: Date; reading: number }>,
): number | null {
  // Rata-rata jam/hari dari 3 bulan terakhir
  const threeMonthsAgo = new Date();
  threeMonthsAgo.setMonth(threeMonthsAgo.getMonth() - 3);

  const recent = hmReadings
    .filter((r) => new Date(r.readingDate) >= threeMonthsAgo)
    .sort(
      (a, b) =>
        new Date(b.readingDate).getTime() - new Date(a.readingDate).getTime(),
    );

  if (recent.length < 2) return null;

  const newest = recent[0];
  const oldest = recent[recent.length - 1];
  const daysDiff =
    (new Date(newest.readingDate).getTime() -
      new Date(oldest.readingDate).getTime()) /
    (1000 * 60 * 60 * 24);
  const hmDiff = newest.reading - oldest.reading;

  const avgHmPerDay = hmDiff / daysDiff;
  if (avgHmPerDay <= 0) return null;

  return Math.round(remainingHours / avgHmPerDay); // hari
}
```

11.7 PCR Forecast — Build Snapshot & Convert to Actual

```ts
// lib/forecasts/build-snapshot.ts
import { prisma } from "@/lib/prisma";
import { calculateComponentLife } from "@/lib/calculations/life";

export async function buildForecastSnapshot(
  fleetEquipmentId: number,
  idMod: number,
) {
  const [equipment, commod, latestHm, lastRep, condition] = await Promise.all([
    prisma.fleetEquipmentCache.findUniqueOrThrow({
      where: { fleetEquipmentId },
    }),
    prisma.commod.findUniqueOrThrow({
      where: { idMod },
      include: { comp: true },
    }),
    prisma.hm.findFirst({
      where: { fleetEquipmentId, deletedAt: null },
      orderBy: { dateHm: "desc" },
    }),
    prisma.replacement.findFirst({
      where: { fleetEquipmentId, idMod, woStatus: "CLOSE", deletedAt: null },
      orderBy: { repDate: "desc" },
    }),
    prisma.condition.findFirst({
      where: { fleetEquipmentId, idMod },
      orderBy: { updatedAt: "desc" },
    }),
  ]);

  const hmNow = Number(latestHm?.hmUnit ?? 0);
  const calc = calculateComponentLife({
    hmNow,
    hmLastReplacement: Number(lastRep?.hmRep ?? 0),
    compHour: lastRep?.compHour ?? 0,
    policy: commod.policy ?? 1,
  });

  return {
    modelName: equipment.modelName,
    unitNo: equipment.unitNo,
    projectCode: equipment.projectCode,
    compDesc: commod.comp.compDesc,
    hmComponent: calc.currentLife,
    policy: commod.policy,
    lifePercent: calc.lifePercent,
    ratingSos: condition?.ratingSos ?? null,
    priceComponent: commod.price,
    snapshotAt: new Date(),
  };
}

// actions/forecasts.ts — convert forecast → replacement WO
export async function convertForecastToReplacement(forecastId: number) {
  const forecast = await prisma.pcrForecast.findUniqueOrThrow({
    where: { idForecast: forecastId },
    include: { equipment: true, commod: { include: { comp: true } } },
```

```
  });

  if (forecast.status !== "OPEN")
    throw new Error("Forecast sudah closed/cancelled");
  if (forecast.baPcrStatus !== "APPROVED")
    throw new Error("BA PCR belum fully approved (tunggu PS → PM/PLM → Direksi)");
  if (forecast.idRep)
    throw new Error("Forecast sudah pernah di-convert");

  const latestHm = await prisma.hm.findFirst({
    where: { fleetEquipmentId: forecast.fleetEquipmentId },
    orderBy: { dateHm: "desc" },
  });

  return prisma.$transaction(async (tx) => {
    const rep = await tx.replacement.create({
      data: {
        repDate: new Date(),
        lastRepDate: null,
        fleetEquipmentId: forecast.fleetEquipmentId,
        idMod: forecast.idMod,
        hmRep: latestHm?.hmUnit ?? forecast.hmComponent,
        lastHmRep: 0,
        woStatus: "OPEN",
        woDate: new Date(),
        compHour: 0,
        compCond: forecast.ratingSos ?? "A",
        remarks: forecast.remark ?? "",
        unitNo: forecast.unitNo,
        projectCode: forecast.projectCode,
      },
    });

    await tx.pcrForecast.update({
      where: { idForecast: forecastId },
      data: { idRep: rep.idRep },
    });

    return rep;
  });
}

// Saat closeReplacement() — sync balik ke forecast jika ada link
export async function syncForecastOnCloseReplacement(
  replacementId: number,
  data: { actionDate: Date; poNumber?: string },
) {
  const forecast = await prisma.pcrForecast.findUnique({
    where: { idRep: replacementId },
  });
  if (!forecast) return;

  await prisma.pcrForecast.update({
    where: { idForecast: forecast.idForecast },
    data: {
      status: "CLOSED",
      actionDate: data.actionDate,
      poNumber: data.poNumber,
    },
```

```
  });
}
```

11.2 Autentikasi & Session

```
// lib/auth.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";
import { prisma } from "./prisma";
import bcrypt from "bcryptjs";

export const { handlers, signIn, signOut, auth } = NextAuth({
  providers: [
    Credentials({
      credentials: {
        username: { label: "Username", type: "text" },
        password: { label: "Password", type: "password" },
      },
      authorize: async (credentials) => {
        const user = await prisma.user.findUnique({
          where: { username: credentials.username as string },
        });
        if (!user || !user.isActive) return null;

        const valid = await bcrypt.compare(
          credentials.password as string,
          user.password,
        );
        if (!valid) return null;

        return {
          id: String(user.id),
          name: user.fullName ?? user.username,
          email: user.email,
          level: user.level,
          sign: user.sign,
          pcrSign: user.pcrSign,
          projectCode: user.projectCode, // langsung string, bukan FK
        };
      },
    }),
  ],
  callbacks: {
    jwt({ token, user }) {
      if (user) {
        token.level = (user as any).level;
        token.sign = (user as any).sign;
        token.pcrSign = (user as any).pcrSign;
        token.projectCode = (user as any).projectCode;
      }
      return token;
    },
    session({ session, token }) {
      session.user.level = token.level as any;
      session.user.sign = token.sign as any;
      session.user.pcrSign = token.pcrSign as any;
```

```
        session.user.projectCode = token.projectCode as any;
        return session;
      },
    },
    pages: {
      signIn: "/login",
    },
  });
```

```
// types/next-auth.d.ts
import { UserLevel, ApprovalSign } from "@prisma/client";
import "next-auth";

declare module "next-auth" {
  interface Session {
    user: {
      id: string;
      name?: string | null;
      email?: string | null;
      level: UserLevel;
      sign: ApprovalSign | null;
      projectCode: string | null; // null = bisa lihat semua (atau sesuai kebijakan)
    };
  }
}
```

11.3 Middleware Route Protection

```
// middleware.ts
import { auth } from "@/lib/auth";
import { NextResponse } from "next/server";

export default auth((req) => {
  const isLoggedIn = !!req.auth;
  const path = req.nextUrl.pathname;
  const isAuthPage = path.startsWith("/login");

  if (!isLoggedIn && !isAuthPage) {
    return NextResponse.redirect(new URL("/login", req.url));
  }

  if (isLoggedIn && isAuthPage) {
    return NextResponse.redirect(new URL("/", req.url));
  }

  const level = req.auth?.user?.level;

  // Admin-only routes
  const adminRoutes = ["/users", "/hour-meters"];
  if (adminRoutes.some((r) => path.startsWith(r)) && level !== "ADMIN") {
    return NextResponse.redirect(new URL("/", req.url));
  }

  // Super User + Admin routes (components, model-components)
```

```
  const superUserRoutes = ["/components", "/model-components"];
  if (superUserRoutes.some((r) => path.startsWith(r)) && level === "USER") {
    return NextResponse.redirect(new URL("/", req.url));
  }
});

export const config = {
  matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],
};
```

11.4 Helper Rating Color (A/B/C/X)

Berguna untuk badge di tabel Vuexy:

```
// lib/utils/rating-color.ts

export type Rating = "A" | "B" | "C" | "X";

const RATING_CONFIG: Record<
  Rating,
  { label: string; color: string; variant: string }
> = {
  A: { label: "A - Good", color: "#28C76F", variant: "success" },
  B: { label: "B - Monitor", color: "#FF9F43", variant: "warning" },
  C: { label: "C - Caution", color: "#EA5455", variant: "danger" },
  X: { label: "X - Critical", color: "#7367F0", variant: "primary" },
};

export function getRatingConfig(rating: Rating) {
  return RATING_CONFIG[rating];
}

// Untuk color coding di ExcelJS
export const RATING_EXCEL_COLOR: Record<Rating, string> = {
  A: "FF28C76F", // hijau
  B: "FFFF9F43", // oranye
  C: "FFEA5455", // merah
  X: "FF7367F0", // ungu
};
```

11.5 Contoh Server Action: Close Replacement WO

```
// actions/replacements.ts
"use server";

import { prisma } from "@/lib/prisma";
import { auth } from "@/lib/auth";
import { calculateComponentLife } from "@/lib/calculations/life";
import { revalidatePath } from "next/cache";

export async function closeReplacement(
  replacementId: number,
  closingHm: number,
) {
```

```
  const session = await auth();
  if (!session) throw new Error("Unauthorized");

  const rep = await prisma.replacement.findUnique({
    where: { id: replacementId },
    include: { modelComponent: true },
  });
  if (!rep || rep.status !== "OPEN")
    throw new Error("WO not found or already closed");

  const calc = calculateComponentLife({
    hmNow: closingHm,
    hmLastReplacement: rep.hmReplacement ?? 0,
    compHour: rep.compHour ?? 0,
    policy: rep.modelComponent.policy,
  });

  await prisma.replacement.update({
    where: { id: replacementId },
    data: {
      status: "CLOSED",
      closedAt: new Date(),
      closedHm: closingHm,
      lifeAchieved: calc.currentLife,
      lifePercent: calc.lifePercent,
    },
  });

  revalidatePath(`/equipments/${rep.fleetEquipmentId}/replacements`);
  return { success: true, lifePercent: calc.lifePercent };
}
```

11.6 Script Migrasi Password Lama

```
// scripts/migrate-passwords.ts
// Jalankan sekali: npx ts-node scripts/migrate-passwords.ts

import { PrismaClient } from "@prisma/client";
import bcrypt from "bcryptjs";

const prisma = new PrismaClient();

async function main() {
  const users = await prisma.user.findMany();
  let migrated = 0;

  for (const user of users) {
    if (!user.password.startsWith("$2")) {
      // belum di-hash
      const hashed = await bcrypt.hash(user.password, 12);
      await prisma.user.update({
        where: { id: user.id },
        data: { password: hashed },
      });
      migrated++;
      console.log(`✓ Hashed: ${user.username}`);
```

```
    }
  }

  console.log(`\nSelesai: ${migrated}/${users.length} password di-hash.`);
}

main().finally(() => prisma.$disconnect());
```

---

12. Keamanan & Auth

Perbaikan dari Legacy

#	Masalah Lama	Solusi Baru
1	Password plaintext	`bcrypt` hash (salt 12)
2	SQL Injection	Prisma ORM (parameterized by default)
3	CSRF disabled	NextAuth built-in CSRF, tambah header
4	XSS disabled	React escape by default + CSP headers
5	Session CI tidak stateless	JWT NextAuth dengan `AUTH_SECRET`
6	File upload tidak divalidasi	Validasi MIME type + max size
7	Endpoint truncate tanpa guard	Dihapus total
8	Role tidak di-check di setiap method	Middleware + check di Server Action

Environment Variables

```
# .env.local — JANGAN commit ke git

# Database lokal ARKA PCR
DATABASE_URL="mysql://root:password@localhost:3306/arka_pcr_new"

# Fleet API
FLEET_API_URL="http://192.168.32.15/ark-fleet/api"

# NextAuth
AUTH_SECRET="generate: openssl rand -base64 32"
AUTH_URL="http://localhost:3000"        # ganti ke domain production

# Upload
UPLOAD_DIR="/var/www/arka-pcr/uploads"  # path absolut di luar web root
MAX_UPLOAD_SIZE_MB="10"
```

Security Headers di `next.config.ts`

```
const nextConfig = {
  async headers() {
    return [
      {
```

```
        source: "/(.*)",
        headers: [
          { key: "X-Frame-Options", value: "DENY" },
          { key: "X-Content-Type-Options", value: "nosniff" },
          { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
          { key: "Permissions-Policy", value: "camera=(), microphone=()" },
        ],
      },
    ];
  },
};

export default nextConfig;
```

---

13. Estimasi Waktu

Phase	Deskripsi	Estimasi
0	Persiapan & backup	1-2 hari
1	Pondasi (Prisma, auth, fleet client)	2-3 hari
2	Fleet API integration + master data PCR	3-4 hari
3	Equipment list (read dari fleet API)	2-3 hari
4	Hour Meter	2-3 hari
5	**PCR Forecast**	5-6 hari
6	PCR / Replacement	4-5 hari
7	SOS	4-5 hari
8	Inspections & Condition	5-6 hari
9	Cannibal & Approval	5-6 hari
10	Dashboard & Laporan	3-4 hari
11	Polish & Deployment	3-4 hari
Total		**~39-51 hari kerja**

> Waktu lebih singkat dari estimasi sebelumnya karena **tidak ada CRUD untuk Project, Unit, dan Model** — ketiganya read-only dari Fleet API. Vuexy template juga sudah menyediakan komponen UI siap pakai.

---

14. Catatan Migrasi Data

Langkah Sebelum Mulai

```
-- Jalankan di DB lama untuk audit schema aktual
SHOW TABLES;
DESCRIBE sos;            -- dapatkan 47+ field aktual
DESCRIBE ba;             -- cek kolom status_l1/l2/l3, user_l1/l2/l3
DESCRIBE user;           -- cek kolom sign
DESCRIBE commod;         -- atau mod — cek nama tabel aktual
```

```
DESCRIBE replacement;    -- cek kolom report, price

-- Mapping unit lama ke fleet equipment ID
SELECT u.id as old_unit_id, u.unit_no, u.id_model
FROM unit u
ORDER BY u.unit_no;
-- Cocokkan unit_no dengan response dari /ark-fleet/api/equipments
```

Mapping Field untuk Migrasi

Tabel Lama	Field Lama	Tabel Baru (Prisma)	Field Baru	Catatan
comp	id_comp	comp	id_comp	Sama
comp	comp_desc	comp	comp_desc	Sama
comp	comp_type	comp	comp_type	Sama
(baru)	—	comp	status	Tambah: "Active" / "Inactive"
mod / commod	id_mod	commod	id_mod	Sama
mod / commod	id_model	commod	id_model (fleetModelId)	Nilai diisi dari model_id Fleet API
mod / commod	id_comp	commod	id_comp	FK ke comp
mod / commod	policy	commod	policy	Sama
(baru)	—	commod	price, life_type	Tambah dari live DB
hm	id_hm	hm	id_hm	Sama
hm	id_unit	hm	id_unit (fleetEquipmentId)	Isi dari id di Fleet API (cocokkan via unit_no)
hm	hm_unit	hm	hm_unit	Sama
hm	wh_day	hm	wh_day	Sama
hm	date_hm	hm	date_hm	Sama
(baru)	—	hm	unit_no, project_code	Denormalized, isi dari data unit
replacement	id_rep	replacement	id_rep	Sama
replacement	rep_date	replacement	rep_date	Sama
replacement	last_rep_date	replacement	last_rep_date	Sama
replacement	id_unit	replacement	id_unit (fleetEquipmentId)	Cocokkan via unit_no ke Fleet API
replacement	id_mod	replacement	id_mod	FK ke commod
replacement	hm_rep, last_hm_rep	replacement	hm_rep, last_hm_rep	Sama

Tabel Lama	Field Lama	Tabel Baru (Prisma)	Field Baru	Catatan
replacement	wo_no, wo_date, wo_status, wo_end_date	replacement	Sama	Sama
replacement	comp_hour, comp_life, life_percent	replacement	Sama	Sama
replacement	comp_cond, remarks	replacement	Sama	Sama
(baru)	—	replacement	report, unit_no, project_code	Tambah dari live DB + denorm
(baru)	—	replacement	back-link dari forecast	pcr_forecast.id_rep → replacement.id_rep
(baru)	Spreadsheet forecast	pcr_forecast	semua kolom snapshot + workflow	Modul baru; import Excel operasional
(baru)	—	pcr_forecast_approval	PS/PM/PLM/OD/FD/PD per role	6 level; gate paralel PM+PLM, OD+FD+PD
user	sign (L1/L2/L3)	user	sign (kanibal) + pcr_sign	Role PCR terpisah dari kanibal
sos	id_sos, id_unit, id_mod	sos	Sama	id_unit → fleetEquipmentId
sos	sample_date, lab_name, lab_no, oil_type	sos	Sama	Sama
sos	h_oil, h_unit, eval_code	sos	Sama	Sama
sos	fe, cu, cr, si, …, visc, tbn, …, dilution	sos	Sama	47 field
sos	4um, 6um, 14um, 15um	sos	p4um, p6um, p14um, p15um	Rename karena Prisma, DB tetap 4um via @map
inspection	id_ins, id_unit, id_mod	inspection	Sama	id_unit → fleetEquipmentId
inspection	type (FC/MPS/VI/TA2/ED)	inspection	type	Sama
inspection	ins_date, ins_hm, rating	inspection	Sama	Sama
condition	id_condition, id_unit, id_mod	condition	Sama	id_unit → fleetEquipmentId

Tabel Lama	Field Lama	Tabel Baru (Prisma)	Field Baru	Catatan
condition	condition	condition	condition + sos_rating, fc_rating, dll.	Rating per sumber ditambah v2
ba	id_project	ba	project_code	Rename kolom INT → VARCHAR kode Fleet API
ba	mr_no, pr_no, po_no	ba	VARCHAR(30)	Was INT, cast saat import
(baru)	—	ba_approval	id_ba, level, status, approved_by, approved_at	Seed dari status_l1/l2/l3 legacy
kanibal	id_rep = 0	kanibal	id_rep = NULL	FK valid ke replacement
kanibal	wo_no_kanibal INT	kanibal	VARCHAR(30)	Was INT
user	id_project	user	project_code	Rename kolom INT → VARCHAR kode Fleet API
(baru)	—	user	full_name, is_active, last_login	Auth modern
(baru)	—	fleet_equipment_cache	snapshot dari Fleet API	Sync periodik
(baru)	—	legacy_unit_mapping	legacy id_unit → fleet id	Migrasi
(baru)	—	semua transaksi	created_at, updated_at, snapshot_at	Audit + snapshot
ba	id_ba, no_ba	ba	Sama	Sama
ba	posting_date, symptom, failure, id_caused, dll.	ba	Sama	Sama
ba	status_l1/l2/l3, user_l1/l2/l3	ba	Sama (flat) + ba_approval	Flat tetap; normalisasi di ba_approval
kanibal	id_kanibal, no_ba, id_rep, id_unit, dll.	kanibal	Sama	id_unit → fleetEquipmentId; id_rep 0→NULL
ba_caused	id_caused, caused	ba_caused	Sama	Sama
ba_action	id_action, action	ba_action	Sama	Sama
ba_status	id_status, status	ba_status	Sama	Sama
user	id_user, username, password	user	Sama	password → bcrypt

Tabel Lama	Field Lama	Tabel Baru (Prisma)	Field Baru	Catatan
user	level (Admin/Super User/User/Guest)	user	level	Sama
user	sign (L1/L2/L3)	user	sign + pcr_sign	Kanibal L1/L2/L3 + role PCR terpisah

Script Mapping Unit Lama → Fleet Equipment ID

```
// scripts/map-units-to-fleet.ts
// Buat file CSV mapping sebelum migrasi data

import { getEquipments } from "@/lib/fleet-api/client";
import { createWriteStream } from "fs";

async function main() {
  const fleetEquipments = await getEquipments();
  const fleetMap = new Map(fleetEquipments.map((e) => [e.unit_no.trim(), e]));

  // Baca dari DB lama (jalankan query manual ke DB lama)
  // const oldUnits = ... (export dari DB lama)

  const out = createWriteStream("mapping-units.csv");
  out.write(
    "old_unit_id,old_unit_no,fleet_equipment_id,fleet_unit_no,matched\n",
  );

  // for (const old of oldUnits) {
  //   const fleet = fleetMap.get(old.unit_no.trim())
  //   out.write(`${old.id},${old.unit_no},${fleet?.id ?? ""},${fleet?.unit_no ??
""},${!!fleet}\n`)
  // }
}

main();
```

Urutan Import Data (Hindari Foreign Key Error)

```
0. fleet_equipment_cache   (sync dari Fleet API — WAJIB sebelum data transaksi)
1. legacy_unit_mapping     (mapping id_unit lama → fleet_equipment_id)
2. user                    (project_code string, tanpa FK ke project)
3. ba_caused, ba_action, ba_status  (lookup)
4. comp
5. commod                  (FK comp; dedupe unique id_model+id_comp dulu)
6. commod                   (FK comp; fleet_model_id dari Fleet API)
7. pcr_forecast              (NEW — FK commod + fleet_equipment_cache; optional id_rep)
8. hm                        (FK fleet_equipment_cache; isi unit_no, project_code,
snapshot_at)
9. replacement               (FK commod + fleet_equipment_cache)
10. sos, inspection          (FK commod + fleet_equipment_cache)
11. condition                (FK commod + fleet_equipment_cache; isi rating per sumber)
12. pcr_forecast_approval    (NEW — FK pcr_forecast + user)
```

```
13. ba                          (FK lookup; project_code dari lookup project lama → kode)
14. ba_approval                 (seed dari status_l1/l2/l3 legacy; FK ba + user)
15. kanibal                     (FK ba.no_ba; id_rep NULL jika legacy=0; FK
fleet_equipment_cache)
```

> **Import spreadsheet forecast historical:** baris `STATUS PCR=Closed` → insert `pcr_forecast` + `replacement` dalam satu transaksi, set `id_rep` dan `action_date`. Baris `Open` → hanya `pcr_forecast`.

Transformasi Wajib Saat Migrasi

```
-- 1. Kanibal: id_rep 0 → NULL
UPDATE kanibal SET id_rep = NULL WHERE id_rep = 0;

-- 2. BA/User: id_project INT → project_code VARCHAR
--    Lookup dari tabel project lama:
--    UPDATE ba SET project_code = (SELECT kode_project FROM project WHERE id_project =
ba.id_project_old);
--    UPDATE user SET project_code = (SELECT kode_project FROM project WHERE id_project =
user.id_project_old);

-- 3. HM/Replacement/SOS: id_unit legacy → fleet_equipment_id via legacy_unit_mapping
--    UPDATE hm h JOIN legacy_unit_mapping m ON h.id_unit = m.legacy_unit_id
--    SET h.id_unit = m.fleet_equipment_id;

-- 4. Commod dedupe: identifikasi duplikat
--    SELECT id_model, id_comp, COUNT(*) FROM commod GROUP BY id_model, id_comp HAVING
COUNT(*) > 1;

-- 5. SOS: oil_change/oil_added 0/1 → boolean
-- 6. WO/MR/PR/PO: INT → VARCHAR (CAST saat import)
-- 7. HM duplikat per tanggal: set is_official = 0 untuk row selain MAX(id_hm) per
(id_unit, date_hm)
```

Perhatian Khusus

- **Fleet API harus bisa diakses** dari server Next.js saat development & production (cek jaringan, firewall)
- **Cocokkan** `unit_no` antara DB lama dengan `unit_no` dari Fleet API sebelum migrasi — kemungkinan ada perbedaan spasi/huruf besar-kecil
- `model_id` **dari Fleet API** tidak sama dengan `id_model` di DB lama — buat tabel mapping sementara
- **SOS fields**: Jalankan `DESCRIBE sos` untuk mendapat field aktual — tambahkan yang kurang di Prisma schema
- **File laporan**: Copy folder `assets/file/` ke `UPLOAD_DIR` baru, path di DB tetap sama
- **Cache Fleet API** jika `/ark-fleet/api` lambat: gunakan `revalidate: 600` (10 menit) atau tambah Redis

---

Dokumen diperbarui: 29 Mei 2026 — Schema v2 + modul PCR Forecast (6-tahap approval organisasi → replacement)
Stack: CodeIgniter 2 (PHP) → Next.js (Vuexy template) + TypeScript + Prisma + MySQL
Master data Project/Equipment/Model: Fleet API `192.168.32.15/ark-fleet/api`